aerospace-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `aerospace_engineering_ai999x.about()` for the same answer.

Every capability aerospace-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import aerospace_engineering_ai999x

result = aerospace_engineering_ai999x.solve('aero_isa_properties', h='0 m')
result.show()
# T = 288.15 K, p = 101325 Pa, rho = 1.225 kg/m^3, a = 340.294 m/s, mu = 1.78938e-05 Pa*s, sigma =
1, delta = 1, theta = 1, H = 0 m
```

By topic

topic	capabilities
Aerodynamics / compressibility	Critical Mach number of an airfoil
Aerodynamics / finite wings	Elliptic wing by lifting-line theory
Aerodynamics / thin-airfoil theory	Thin-airfoil theory for a NACA 4-digit airfoil
Aircraft performance / ceilings	Absolute ceiling of a jet aircraft, Service ceiling of a jet aircraft
Aircraft performance / climb	Time to climb with a linearly decreasing rate of climb, Maximum rate of climb of a jet, Steepest climb angle of a jet, Maximum rate of climb of a propeller aircraft
Aircraft performance / drag polar	Best lift coefficients and lift-to-drag ratios of a drag polar, Induced drag in level flight
Aircraft performance / gliding flight	Glide ratio, angle and range
Aircraft performance / level flight	Speeds for minimum drag, minimum power and best jet range, Stall speed in level flight, Lift coefficient in level flight, Wing loading
Aircraft performance / navigation	Wind triangle: heading and ground speed
Aircraft performance / range and endurance	Breguet range of a jet aircraft, Endurance of a jet aircraft, Fuel a jet needs for a cruise range, Jet range at constant altitude and lift coefficient, Breguet range of a propeller aircraft, Endurance of a propeller aircraft at constant altitude
Aircraft performance / takeoff and landing	Takeoff ground roll, Landing ground roll

topic	capabilities
Aircraft performance / turning flight	Level coordinated turn at a bank angle, Level turn at a load factor
Airspeeds / calibrated airspeed	Calibrated airspeed from true airspeed
Airspeeds / pitot-static Mach meter	Mach number from pitot and static pressures
Airspeeds / ram temperature rise	Total (stagnation) air temperature in flight
Airspeeds / true airspeed	True airspeed from calibrated airspeed
Compressible flow / expansion waves	Prandtl-Meyer angle of a Mach number, Mach number from the Prandtl-Meyer angle, Prandtl-Meyer expansion fan
Compressible flow / Fanno flow	Fanno flow (adiabatic duct flow with friction)
Compressible flow / isentropic flow	Isentropic flow ratios at a Mach number, Mach number from the stagnation-to-static pressure ratio
Compressible flow / normal shocks	Normal shock relations, Normal shock from its pressure ratio
Compressible flow / nozzle area-Mach relation	Mach number from the area ratio A/A*
Compressible flow / oblique shocks	Oblique shock from the deflection angle, Oblique shock from the wave angle, Maximum deflection angle of an attached oblique shock
Compressible flow / Rayleigh flow	Rayleigh flow (duct flow with heat addition)
Nozzles / choked flow	Choked mass flow through a nozzle throat
Nozzles / off-design operation	Normal shock position in a converging-diverging nozzle
Orbits / circular orbits	Circular orbit at an altitude above the Earth, Circular-orbit altitude for an orbital period
Orbits / escape	Escape speed at an altitude above the Earth
Orbits / orbit determination and targeting	Lambert's problem (orbit through two positions in a given time)
Orbits / orbital transfers	Bi-elliptic transfer between circular orbits
Orbits / perturbations	Nodal regression from the Earth's oblateness (J2), Rotation of the line of apsides from J2, Inclination of a sun-synchronous orbit
Orbits / satellite coverage	Coverage of a satellite at an altitude
Propellers and rotors / momentum theory	Ideal hover power of a rotor (momentum theory), Ideal propeller in axial flight (momentum theory)
Re-entry / ballistic coefficient	Ballistic coefficient
Re-entry / ballistic entry	Peak deceleration of a ballistic re-entry (Allen-Eggers), Speed and deceleration of a ballistic re-entry at an altitude
Rocket performance / gravity losses	Gravity loss of a vertical rocket burn
Rocket performance / rocket equation	Rocket delta-v from specific impulse (Tsiolkovsky), Propellant mass for a required delta-v

topic	capabilities
Rocket performance / sounding rockets	Burnout height and apogee of a vertical rocket
Rocket performance / staging	Delta-v of a multistage rocket, Payload fraction of a rocket with identical stages, Optimal staging of a multistage rocket
Rocket propulsion / characteristic velocity	Characteristic velocity c* of a rocket propellant
Rocket propulsion / engine performance	Ideal rocket engine performance
Rocket propulsion / engine sizing	Throat area of a rocket engine for a required thrust
Rocket propulsion / exhaust velocity	Ideal rocket exhaust velocity
Rocket propulsion / nozzle design	Optimum nozzle area ratio
Rocket propulsion / thrust coefficient	Nozzle thrust coefficient
Rockets / rocket engine performance	Rocket thrust and specific impulse from mass flow and exhaust velocity
Stability / longitudinal static stability	Neutral point and static margin (wing and tail)
Standard atmosphere / density altitude	Density altitude from pressure altitude and temperature
Standard atmosphere / flight condition	Mach number, dynamic pressure, equivalent airspeed and Reynolds number at an altitude
Standard atmosphere / geopotential height	Geopotential height from geometric altitude
Standard atmosphere / ISA properties	Standard atmosphere at an altitude
Standard atmosphere / pressure altitude	Pressure altitude from static pressure

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()

outcome	meaning	what to do
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
aero_isa_properties	Standard atmosphere at an altitude	h	T, p, rho, a, mu, sigma, delta, theta, H
aero_pressure_altitude	Pressure altitude from static pressure	p	h, h_ft, H, T_std
aero_density_altitude	Density altitude from pressure altitude and temperature	hp, T	rho, sigma, h_rho, h_rho_ft, p
aero_geopotential_altitude	Geopotential height from geometric altitude	h	H, g, dH
aero_calibrated_airspeed	Calibrated airspeed from true airspeed	V, hp, T	CAS, CAS_kt, qc, pt
aero_true_airspeed_from_calibrated	True airspeed from calibrated airspeed	CAS, hp, T	TAS, TAS_kt, qc, M
aero_mach_from_pitot	Mach number from pitot and static pressures	p, pt, gamma	M

id	operation	inputs	outputs
aero_isentropic_flow	Isentropic flow ratios at a Mach number	M, gamma	T0_T, p0_p, rho0_rho, A_Astar
aero_mach_from_area_ratio	Mach number from the area ratio A/A*	ar, regime, gamma	M, p_p0, T_T0
aero_mach_from_pressure_ratio	Mach number from the stagnation-to-static pressure ratio	pr, gamma	M, T0_T, A_Astar
aero_normal_shock	Normal shock relations	M1, gamma	M2, p2_p1, T2_T1, rho2_rho1, p02_p01, p02_p1
aero_oblique_shock	Oblique shock from the deflection angle	M1, theta, branch, gamma	beta_deg, M2, p2_p1, T2_T1, rho2_rho1, p02_p01, Mn1
aero_prandtl_meyer	Prandtl-Meyer angle of a Mach number	M, gamma	nu_deg
aero_mach_from_prandtl_meyer	Mach number from the Prandtl-Meyer angle	nu, gamma	M, nu_max_deg
aero_expansion_fan	Prandtl-Meyer expansion fan	M1, theta, gamma	M2, nu1_deg, nu2_deg, p2_p1, T2_T1, rho2_rho1
aero_normal_shock_from_pressure_ratio	Normal shock from its pressure ratio	p21, gamma	M1, M2, T2_T1, rho2_rho1, p02_p01
aero_oblique_shock_from_wave_angle	Oblique shock from the wave angle	M1, beta, gamma	theta_deg, M2, p2_p1, T2_T1, rho2_rho1, p02_p01, Mn1
aero_max_deflection_angle	Maximum deflection angle of an attached oblique shock	M1, gamma	theta_max_deg, beta_deg, M2, p2_p1
aero_fanno_flow	Fanno flow (adiabatic duct flow with friction)	M, gamma	fLD, p_pstar, T_Tstar, rho_rhostar, p0_p0star
aero_rayleigh_flow	Rayleigh flow (duct flow with heat addition)	M, gamma	T0_T0star, T_Tstar, p_pstar, rho_rhostar, p0_p0star
aero_choked_mass_flow	Choked mass flow through a nozzle throat	p0, T0, A, gamma, R	mdot, p_star, T_star, V_star
aero_characteristic_velocity	Characteristic velocity c* of a rocket propellant	Tc, Mw, gamma	c_star, Gamma
aero_thrust_coefficient	Nozzle thrust coefficient	gamma, eps, pc, pa	CF, pe_pc, Me, CF_vac
aero_ideal_exhaust_velocity	Ideal rocket exhaust velocity	Tc, Mw, gamma, pc, pe	ve, Isp, Me
aero_optimum_area_ratio	Optimum nozzle area ratio	gamma, pc, pa	eps, Me, CF
aero_rocket_engine_performance	Ideal rocket engine performance	pc, Tc, Mw, gamma, At, eps, pa	F, mdot, Isp, CF, c_star, pe, ve, Me
aero_throat_area_for_thrust	Throat area of a rocket engine for a required thrust	F, pc, Tc, Mw, gamma, eps, pa	At, Ae, mdot, Isp
aero_nozzle_normal_shock	Normal shock position in a converging-diverging nozzle	pc, pb, eps, gamma	As_At, M1, M2, Me, p02_p01
aero_critical_mach	Critical Mach number of an airfoil	Cp0, rule	M_cr, Cp_cr

id	operation	inputs	outputs
aero_rocket_delta_v	Rocket delta-v from specific impulse (Tsiolkovsky)	Isp, m0, mf	dv
aero_propellant_for_delta_v	Propellant mass for a required delta-v	dv, Isp, mf	m0, mp, R
aero_delta_v_gravity_loss	Gravity loss of a vertical rocket burn	Isp, m0, mf, tb, g	v_b, dv_ideal, loss
aero_vertical_rocket_trajectory	Burnout height and apogee of a vertical rocket	Isp, m0, mf, tb, g	v_b, h_b, h_max, t_apex
aero_multistage_delta_v	Delta-v of a multistage rocket	Isp, m0, mf	dv, dv_stages
aero_identical_stages_payload	Payload fraction of a rocket with identical stages	n, dv, Isp, eps	lam, payload_fraction, R, gross_per_payload
aero_circular_orbit	Circular orbit at an altitude above the Earth	h	v, T, T_min, revs_per_day
aero_escape_speed	Escape speed at an altitude above the Earth	h	v_esc, v_circ, dv
aero_altitude_for_period	Circular-orbit altitude for an orbital period	T	h, h_km, v
aero_bielliptic_transfer	Bi-elliptic transfer between circular orbits	h1, h2, hb	dv1, dv2, dv3, dv, t, t_days, dv_hohmann
aero_lambert_problem	Lambert's problem (orbit through two positions in a given time)	r1, r2, tof, direction	v1, v2, dtheta_deg
aero_j2_nodal_regression	Nodal regression from the Earth's oblateness (J2)	hp, i, ha	raan_rate_deg_day, raan_rate
aero_j2_apsidal_rotation	Rotation of the line of apsides from J2	hp, ha, i	argp_rate_deg_day, argp_rate
aero_sun_synchronous_inclination	Inclination of a sun-synchronous orbit	h	i_deg, period_min
aero_ballistic_entry_peak_deceleration	Peak deceleration of a ballistic re-entry (Allen-Eggers)	m, CD, A, VE, gamma_e, H, rho0	n_max, a_max, h_peak, V_peak
aero_ballistic_entry_velocity	Speed and deceleration of a ballistic re-entry at an altitude	m, CD, A, VE, gamma_e, h, H, rho0	V, n, a
aero_drag_polar_optimum	Best lift coefficients and lift-to-drag ratios of a drag polar	CD0, AR, e	CL_md, LD_max, CL_mp, LD_mp, CL_mr, LD_mr
aero_min_drag_power_speeds	Speeds for minimum drag, minimum power and best jet range	W, S, h, CD0, AR, e	V_md, D_min, V_mp, P_min, V_mr
aero_breguet_range_jet	Breguet range of a jet aircraft	V, c, LD, W0, W1	R, R_km, t, t_h
aero_breguet_endurance_jet	Endurance of a jet aircraft	c, LD, W0, W1	E, E_h
aero_jet_fuel_for_range	Fuel a jet needs for a cruise range	R, V, c, LD, W0	W1, Wf, mf
aero_jet_range_constant_altitude	Jet range at constant altitude and lift coefficient	W0, W1, S, h, CL, CD, c	R, R_km, t, t_h
aero_breguet_range_prop	Breguet range of a propeller aircraft	eta, c, LD, W0, W1	R, R_km

id	operation	inputs	outputs
aero_prop_endurance_constant_altitude	Endurance of a propeller aircraft at constant altitude	eta, c, CL, CD, S, h, W0, W1	E, E_h
aero_takeoff_ground_roll	Takeoff ground roll	W, S, h, T, mu, CL, CD, VLO	s, t
aero_landing_ground_roll	Landing ground roll	W, S, h, VTD, mu, CL, CD, Tr	s, t
aero_level_turn_bank	Level coordinated turn at a bank angle	V, phi, g	n, phi_deg, R, omega, omega_deg, t_360
aero_level_turn_load_factor	Level turn at a load factor	V, n, g	n, phi_deg, R, omega, omega_deg, t_360
aero_time_to_climb	Time to climb with a linearly decreasing rate of climb	RC0, Hc, h1, h2	t, t_min
aero_max_rate_of_climb_jet	Maximum rate of climb of a jet	W, S, h, T, CD0, AR, e	RC_max, V, gamma_deg
aero_max_climb_angle_jet	Steepest climb angle of a jet	W, S, h, T, CD0, AR, e	gamma_max_deg, V, RC
aero_max_rate_of_climb_prop	Maximum rate of climb of a propeller aircraft	W, S, h, P, eta, CD0, AR, e	RC_max, V, gamma_deg
aero_absolute_ceiling_jet	Absolute ceiling of a jet aircraft	W, S, T0, CD0, AR, e	h_abs, h_abs_ft, sigma
aero_service_ceiling_jet	Service ceiling of a jet aircraft	W, S, T0, CD0, AR, e, RCs	h_s, h_s_ft
aero_neutral_point	Neutral point and static margin (wing and tail)	h_ac, a_w, a_t, St_S, lt_c, deps, h_cg, eta_t	h_n, SM, CL_alpha, Cm_alpha
aero_actuator_disk_hover	Ideal hover power of a rotor (momentum theory)	T, D, rho	v_i, P_ideal, w
aero_actuator_disk_forward	Ideal propeller in axial flight (momentum theory)	T, V, D, rho	v_i, P_ideal, eta_ideal, w
aero_naca4_thin_airfoil	Thin-airfoil theory for a NACA 4-digit airfoil	m, p, alpha	alpha_L0_deg, CL, Cm_c4, Cm_le
aero_elliptic_wing_lifting_line	Elliptic wing by lifting-line theory	AR, alpha, a0	CL_alpha, CL, CDi, alpha_i_deg
aero_wind_triangle	Wind triangle: heading and ground speed	TAS, course, wind_from, wind_speed	heading_deg, wca_deg, GS, GS_kt
aero_total_temperature	Total (stagnation) air temperature in flight	V, T	T0, dT, M
aero_optimal_staging	Optimal staging of a multistage rocket	Isp, eps, dv, m_pl	m0, payload_fraction, R, stage_mass
aero_satellite_coverage	Coverage of a satellite at an altitude	h, eps_min	lambda_deg, nadir_deg, slant_range, coverage
aero_stall_speed	Stall speed in level flight	W, S, CLmax, h	V_stall, V_stall_kt, EAS_stall, wing_loading
aero_level_flight_lift_coefficient	Lift coefficient in level flight	W, S, V, h	CL, q, wing_loading
aero_wing_loading	Wing loading	W, S	wing_loading, wing_loading_kg_m2,

id	operation	inputs	outputs
			wing_loading_psf
aero_induced_drag	Induced drag in level flight	W, S, V, h, AR, e	CL, CDi, Di, P_induced
aero_glide_range	Glide ratio, angle and range	dh, LD, V	range, range_km, glide_angle_deg, sink_rate, time
aero_rocket_thrust	Rocket thrust and specific impulse from mass flow and exhaust velocity	mdot, ve, pe, pa, Ae	F, F_kN, Isp, c_eff, F_pressure
aero_ballistic_coefficient	Ballistic coefficient	m, CD, A	beta, beta_psf
aero_flight_condition	Mach number, dynamic pressure, equivalent airspeed and Reynolds number at an altitude	V, h, L	M, q, EAS, EAS_kt, a, rho, Re

Standard atmosphere at an altitude — `aero_isa_properties`

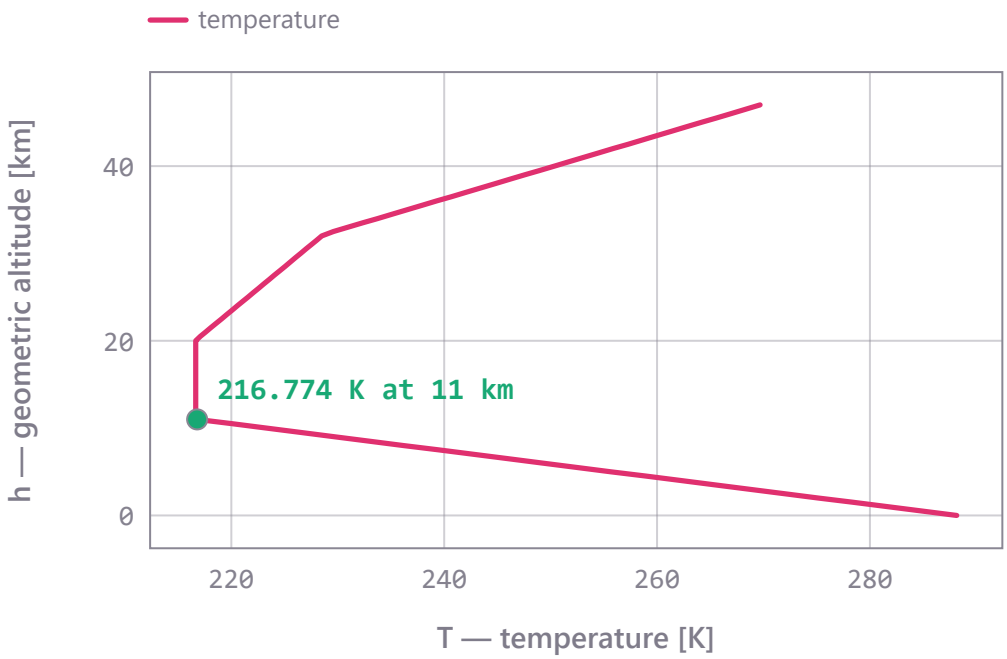
Temperature, pressure, density, speed of sound and viscosity of the International Standard Atmosphere (US 1976) at a geometric altitude, with the density, pressure and temperature ratios and the geopotential height.

$$T = T_b + L(H - H_b), \quad p = p_b\left(\frac{T_b}{T}\right)^{g_0/(RL)}, \quad \rho = \frac{p}{RT}, \quad a = \sqrt{\gamma RT}$$

input	meaning	condition
`h`	geometric altitude above mean sea level (m, km, ft), -1 km to 47 km	

output	meaning
`T`	temperature
`p`	pressure
`rho`	density
`a`	speed of sound
`mu`	dynamic viscosity (Sutherland)
`sigma`	density ratio $\rho/rho0$
`delta`	pressure ratio p/p_0
`theta`	temperature ratio T/T_0
`H`	geopotential height

- **Method:** $H = r_0\,h/(r_0 + h)$; in each layer $T = Tb + L$ (H - Hb); $p = pb$ (Tb/T)^(g0/(R L)) $(L \neq 0)$ or pb exp(-g0 (H - Hb)/(R Tb)); rho = p/(R T); $a = sqrt$(gamma R T)
- **Assumes:** -1000 $m \leq h \leq 47000$ m
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Temperature of the International Standard Atmosphere from 0 to 47 km, 95 altitudes computed by the package (it covers the atmosphere to 47 km): 288.15 K at sea level, the tropospheric lapse to 216.774 K at 11 km (marked), the isothermal layer (coldest computed point 216.65 K at 11.5 km) and the warming stratosphere, 269.684 K at 47 km.

```
aerospace_engineering_ai999x.solve('aero_isa_properties', h=0.0)
# -> T = 288.15 K, p = 101325 Pa, rho = 1.225 kg/m^3, a = 340.294 m/s, mu = 1.78938e-05 Pa*s, sigma
= 1, delta = 1, theta = 1, H = 0 m   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_isa_properties', h=11019.067832000108)
# -> T = 216.65 K, p = 22632.1 Pa, rho = 0.363918 kg/m^3, a = 295.07 m/s, mu = 1.42161e-05 Pa*s,
sigma = 0.297076, delta = 0.223361, theta = 0.751865, H = 11000 m   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_isa_properties', h=5003.93591325625)
# -> T = 255.65 K, p = 54019.9 Pa, rho = 0.736115 kg/m^3, a = 320.53 m/s, mu = 1.62812e-05 Pa*s,
sigma = 0.600911, delta = 0.533135, theta = 0.887212, H = 5000 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ISA properties: h = 0 m	SOLVED	T = 288.15 K, p = 101325 Pa, rho = 1.225 kg/m^3, a = 340.294 m/s, mu = 1.78938e-05 Pa*s, sigma = 1, delta = 1, theta = 1, H = 0 m
Standard atmosphere at altitude: h = 15 km	SOLVED	T = 216.65 K, p = 12111.8 Pa, rho = 0.194755 kg/m^3, a = 295.07 m/s, mu = 1.42161e-05 Pa*s, sigma = 0.158984, delta = 0.119534, theta = 0.751865, H = 14964.7 m
ISA properties: h = 60 km	INVALID_INPUT	Standard atmosphere at an altitude: the altitude must lie between -1 km and 47 km (standard atmosphere layers to 47 km).
ISA properties: h = 3000	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: aerospace_engineering_ai999x.temp
ISA properties:	NEEDS_INPUT	aero_isa_properties needs h (geometric altitude above mean sea level (m, km, ft), -1 km to 47 km). Example: solve('aero_

Pressure altitude from static pressure — `aero_pressure_altitude`

The pressure altitude: the height in the standard atmosphere at which the standard pressure equals the measured static pressure (what an altimeter set to 1013.25 hPa reads).

$$H = H_b + \frac{T_b}{L}\left[\left(\frac{p}{p_b}\right)^{-RL/g_0} - 1\right], \quad h = \frac{r_0 H}{r_0 - H}$$

input	meaning	condition
`p`	static pressure (Pa, hPa, inHg, psi)	

output	meaning
`h`	pressure altitude (geometric)
`h_ft`	pressure altitude in feet
`H`	geopotential pressure altitude
`T_std`	standard temperature at that altitude

- **Method:** invert p(H) layer by layer: $H = Hb + (T - Tb)/L$ with $T = Tb$ (p/pb)^(-R L/g0), or H = Hb + (R Tb/g0) ln(pb/p) in an isothermal layer; $h = r_0\, H/(r_0 - H)$
- **Assumes:** p within the standard atmosphere between -1 km and 47 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_pressure_altitude', p=101325.0)
# -> h = 0 m, h_ft = 0, H = 0 m, T_std = 288.15 K   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_pressure_altitude', p=22632.06397346291)
# -> h = 11019.1 m, h_ft = 36151.8, H = 11000 m, T_std = 216.65 K   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_pressure_altitude', p=5474.8886696777745)
# -> h = 20063.1 m, h_ft = 65823.9, H = 20000 m, T_std = 216.65 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pressure altitude: p = 1013.25 hPa	SOLVED	h = 0 m, h_ft = 0, H = 0 m, T_std = 288.15 K
Pressure altitude: p = 101325 Pa	SOLVED	h = 0 m, h_ft = 0, H = 0 m, T_std = 288.15 K
Pressure altitude: p = 0.5 hPa	INVALID_INPUT	Pressure altitude from static pressure: the pressure must lie within the standard atmosphere from -1 km to 47 km.
Pressure altitude: p = 500	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: aerospace_engineering_ai999x.temp
Pressure altitude:	NEEDS_INPUT	aero_pressure_altitude needs p (static pressure (Pa, hPa, inHg, psi)). Example: solve('aero_pressure_altitude', p='10132

Density altitude from pressure altitude and temperature — `aero_density_altitude`

Air density and density altitude (the standard-atmosphere height with the same density) from the pressure altitude and the outside air temperature.

$$\rho = \frac{p_{std}(h_p)}{RT}, \quad \rho_{std}(h_\rho) = \rho$$

input	meaning	condition
`hp`	pressure altitude (m, ft)	
`T`	outside air temperature (K, °C, °F)	

output	meaning
`rho`	air density
`sigma`	density ratio $\rho/rho0$
`h_rho`	density altitude (geometric)
`h_rho_ft`	density altitude in feet
`p`	static pressure at the pressure altitude

- **Method:** p = p_std(hp); rho = p/(R T); density altitude = the standard height where rho_std(h) = rho (inverted layer by layer)
- **Assumes:** 150 $K \le T \le 350$ K; -1 $km \le hp \le 47$ km; the density within the standard atmosphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_density_altitude', hp=0.0, T=288.15)
# -> rho = 1.225 kg/m^3, sigma = 1, h_rho = 0 m, h_rho_ft = 0, p = 101325 Pa   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_density_altitude', hp=11019.067832000108, T=216.65)
# -> rho = 0.363918 kg/m^3, sigma = 0.297076, h_rho = 11019.1 m, h_rho_ft = 36151.8, p = 22632.1 Pa
(SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_density_altitude', hp=0.0, T=300.0)
# -> rho = 1.17661 kg/m^3, sigma = 0.9605, h_rho = 417.839 m, h_rho_ft = 1370.86, p = 101325 Pa
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Density altitude: hp = 0 ft, T = 15 °C	SOLVED	rho = 1.225 kg/m^3, sigma = 1, h_rho = 0 m, h_rho_ft = 0, p = 101325 Pa
Density altitude: hp = 0 m, T = 300 K	SOLVED	rho = 1.17661 kg/m^3, sigma = 0.9605, h_rho = 417.839 m, h_rho_ft = 1370.86, p = 101325 Pa
Density altitude: hp = 2000 m, T = 500 K	INVALID_INPUT	Density altitude from pressure altitude and temperature: the outside air temperature must lie between 150 K and 350 K.
Density altitude: hp = 2000 m	NEEDS_INPUT	aero_density_altitude needs T (outside air temperature (K, °C, °F)). Example: solve('aero_density_altitude', hp='0 m', T

Geopotential height from geometric altitude — `aero_geopotential_altitude`

The geopotential height (the height in a uniform gravity field g0 with the same potential energy) and the local gravity at a geometric altitude.

$$H = \frac{1}{g_0}\int_0^h g\,dz = \frac{r_0 h}{r_0 + h}, \quad g = g_0\left(\frac{r_0}{r_0 + h}\right)^2$$

input	meaning	condition
`h`	geometric altitude (m, km, ft), 0 to 1000 km	

output	meaning
`H`	geopotential height
`g`	gravitational acceleration at the altitude
`dH`	geometric minus geopotential height

- **Method:** $H = (1/g_0)$ integral of g $dz = r_0\,h/(r_0 + h)$; $g = g_0$ (r0/(r0 + h))^2
- **Assumes:** $0 \le h \le 1000$ km; inverse-square gravity with $r_0 = 6356766$ m
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_geopotential_altitude', h=0.0)
# -> H = 0 m, g = 9.80665 m/s^2, dH = 0 m   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_geopotential_altitude', h=706307.3333333334)
# -> H = 635677 m, g = 7.94339 m/s^2, dH = 70630.7 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Geopotential height: h = 0 km	SOLVED	H = 0 m, g = 9.80665 m/s^2, dH = 0 m
Geopotential height: h = 706307.3333333334 m	SOLVED	H = 635677 m, g = 7.94339 m/s^2, dH = 70630.7 m
Geopotential height: h = -5 km	INVALID_INPUT	Geopotential height from geometric altitude: the altitude must lie between 0 and 1000 km.
Geopotential height:	NEEDS_INPUT	aero_geopotential_altitude needs h (geometric altitude (m, km, ft), 0 to 1000 km). Example: solve('aero_geopotential_alt

Calibrated airspeed from true airspeed — `aero_calibrated_airspeed`

The calibrated airspeed, impact pressure and pitot total pressure of an aircraft flying at a true airspeed at a pressure altitude (subsonic, compressible pitot formula).

$$q_c = p\left[\left(1 + 0.2M^2\right)^{3.5} - 1\right], \quad V_c = a_0\sqrt{5\left[\left(\frac{q_c}{p_0} + 1\right)^{2/7} - 1\right]}$$

input	meaning	condition
`V`	true airspeed (m/s, km/h, kt)	
`hp`	pressure altitude (m, ft)	
`T`	outside air temperature (default: standard temperature)	

output	meaning
`CAS`	calibrated airspeed
`CAS_kt`	calibrated airspeed in knots
`qc`	impact pressure pt - p
`pt`	pitot total pressure

- **Method:** $M = V/sqrt$(gamma R T); $qc = p$[(1 + 0.2 M^2)^3.5 - 1]; CAS = a0 sqrt(5[$(qc/p_0 + 1)^{2/7} - 1$])
- **Assumes:** $V > 0$; flight Mach number below 1; -1 $km \le hp \le 47$ km; 150 $K \le T \le 350$ K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_calibrated_airspeed', V=100.0, hp=0.0)
# -> CAS = 100 m/s, CAS_kt = 194.384, qc = 6258.37 Pa, pt = 107583 Pa   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_calibrated_airspeed', V=200.0, hp=0.0)
# -> CAS = 200 m/s, CAS_kt = 388.769, qc = 26689.4 Pa, pt = 128014 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Calibrated airspeed: V = 150 kt, hp = 0 ft	SOLVED	CAS = 77.1667 m/s, CAS_kt = 150, qc = 3694.38 Pa, pt = 105019 Pa
Calibrated airspeed: V = 250 m/s, hp = 0 m	SOLVED	CAS = 250 m/s, CAS_kt = 485.961, qc = 43729 Pa, pt = 145054 Pa
Calibrated airspeed: V = 400 m/s, hp = 0 m	INVALID_INPUT	Calibrated airspeed from true airspeed: the flight Mach number must be subsonic (below 1) for the subsonic pitot formula
Calibrated airspeed: V = 250 m/s	NEEDS_INPUT	aero_calibrated_airspeed needs hp (pressure altitude (m, ft)). Example: solve('aero_calibrated_airspeed', V='100 m/s', h

True airspeed from calibrated airspeed — `aero_true_airspeed_from_calibrated`

The true airspeed and flight Mach number from the calibrated airspeed, the pressure altitude and the outside air temperature (subsonic).

$$q_c = p_0 \left[\left(1 + 0.2 \frac{V_c^2}{a_0^2} \right)^{3.5} - 1 \right], \quad M = \sqrt{5 \left[\left(\frac{q_c}{p} + 1 \right)^{2/7} - 1 \right]}, \quad V = Ma$$

input	meaning	condition
`CAS`	calibrated airspeed (m/s, kt, km/h)	

input	meaning	condition
`hp`	pressure altitude (m, ft)	
`T`	outside air temperature (default: standard temperature)	

output	meaning
`TAS`	true airspeed
`TAS_kt`	true airspeed in knots
`qc`	impact pressure
`M`	flight Mach number

- **Method:** $qc = p_0$[(1 + 0.2 (CAS/a0)^2)^3.5 - 1]; $M = sqrt$(5[$(qc/p + 1)^{2/7} - 1$]); TAS = M sqrt(gamma R T)
- **Assumes:** 0 < CAS < a0; subsonic flight; -1 $km \leq hp \leq 47$ km; 150 $K \leq T \leq 350$ K
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_true_airspeed_from_calibrated', CAS=100.0, hp=0.0)
# -> TAS = 100 m/s, TAS_kt = 194.384, qc = 6258.37 Pa, M = 0.293863   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_true_airspeed_from_calibrated', CAS=150.0, hp=0.0)
# -> TAS = 150 m/s, TAS_kt = 291.577, qc = 14463.7 Pa, M = 0.440795   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
True airspeed: CAS = 120 kt, hp = 0 ft	SOLVED	TAS = 61.7333 m/s, TAS_kt = 120, qc = 2353.51 Pa, M = 0.181412
True airspeed from calibrated airspeed: CAS = 80 m/s, hp = 0 m	SOLVED	TAS = 80 m/s, TAS_kt = 155.508, qc = 3974.46 Pa, M = 0.235091
True airspeed: CAS = 400 m/s, hp = 0 m	INVALID_INPUT	True airspeed from calibrated airspeed: the calibrated airspeed must be below the sea-level speed of sound (subsonic pit
True airspeed: CAS = 250 kt	NEEDS_INPUT	aero_true_airspeed_from_calibrated needs hp (pressure altitude (m, ft)). Example: solve('aero_true_airspeed_from_calibra

Mach number from pitot and static pressures — `aero_mach_from_pitot`

The flight Mach number from the static pressure and the pitot (total) pressure: the isentropic relation when subsonic, the Rayleigh pitot formula (normal shock ahead of the probe) when supersonic.

$$\frac{p_{t2}}{p_1} = \left[\frac{(\gamma+1)^2 M^2}{4\gamma M^2 - 2(\gamma-1)}\right]^{\frac{\gamma}{\gamma-1}} \frac{1-\gamma+2\gamma M^2}{\gamma+1}$$

input	meaning	condition
`p`	static pressure	
`pt`	pitot (total) pressure measured by the probe	

input	meaning	condition
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M`	flight Mach number

- **Method:** subsonic: pt/p = (1 + (g-1)/2 $M^2)^(g/(g-1))$; supersonic: Rayleigh pitot formula $pt2/p_1$ = [$(g+1)^2$ M^2/(4 g M^2 − 2(g-1))]^(g/(g-1)) (1 - g + 2 g M^2)/(g+1), solved by bisection
- **Assumes:** $pt > p > 0$; $M \leq 20$; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_mach_from_pitot', p=10000.0, pt=18929.29158737854)
# -> M = 1   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_mach_from_pitot', p=10000.0, pt=13959.645769144721)
# -> M = 0.707107   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_mach_from_pitot', p=10000.0, pt=56404.40812823316)
# -> M = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mach from pitot: p = 10 kPa, pt = 18.92929158737854 kPa	SOLVED	M = 1
Mach from pitot: p = 1 bar, pt = 12.060964701266624 bar	SOLVED	M = 3
Mach from pitot: p = 50 kPa, pt = 40 kPa	INVALID_INPUT	Mach number from pitot and static pressures: the pitot (total) pressure must exceed the static pressure.
Mach from pitot: p = 50 kPa	NEEDS_INPUT	aero_mach_from_pitot needs pt (pitot (total) pressure measured by the probe). Example: solve('aero_mach_from_pitot', p='

Isentropic flow ratios at a Mach number — `aero_isentropic_flow`

Stagnation-to-static temperature, pressure and density ratios and the area ratio A/Aof isentropic flow of a perfect gas at a Mach number.

$$\frac{T_0}{T} = 1 + \frac{\gamma - 1}{2} M^2, \quad \frac{p_0}{p} = \left(\frac{T_0}{T}\right)^{\frac{\gamma}{\gamma-1}}, \quad \frac{A}{A^*} = \frac{1}{M}\left[\frac{2}{\gamma+1}\frac{T_0}{T}\right]^{\frac{\gamma+1}{2(\gamma-1)}}$$

input	meaning	condition
`M`	Mach number (0.01 to 10)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`T0_T`	stagnation-to-static temperature ratio T_0/T

output	meaning
`p0_p`	stagnation-to-static pressure ratio p_0/p
`rho0_rho`	stagnation-to-static density ratio $rho0/\rho$
`A_Astar`	area ratio A/A* (A* = sonic throat area)

- **Method:** $T_0/T = 1 + (g-1)/2\,M^2$; p0/p = (T0/T)^(g/(g-1)); rho0/rho = (T0/T)^(1/(g-1)); A/A* = $(1/M)$[(2/(g+1))(T_0/T)]^((g+1)/(2(g-1)))
- **Assumes:** $0.01 \le M \le 10$; $1.05 \le \gamma \le 1.7$; isentropic flow of a calorically perfect gas
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_isentropic_flow', M=2.0)
# -> T0_T = 1.8, p0_p = 7.82445, rho0_rho = 4.34692, A_Astar = 1.6875   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_isentropic_flow', M=1.0, gamma=1.3)
# -> T0_T = 1.15, p0_p = 1.83242, rho0_rho = 1.5934, A_Astar = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Isentropic flow: M = 2	SOLVED	T0_T = 1.8, p0_p = 7.82445, rho0_rho = 4.34692, A_Astar = 1.6875
Isentropic flow: M = 1, gamma = 1.4	SOLVED	T0_T = 1.2, p0_p = 1.89293, rho0_rho = 1.57744, A_Astar = 1
Isentropic flow: M = 2, gamma = 3	INVALID_INPUT	Isentropic flow ratios at a Mach number: the ratio of specific heats gamma must lie between 1.05 and 1.7.
Isentropic flow: gamma = 1.4	NEEDS_INPUT	aero_isentropic_flow needs M (Mach number (0.01 to 10)). Example: solve('aero_isentropic_flow', M=2).

Mach number from the area ratio A/A* — `aero_mach_from_area_ratio`

The subsonic or supersonic Mach number at which isentropic flow has a given area ratio A/A*, with the static-to-stagnation pressure and temperature ratios there.

$$\frac{A}{A^*} = \frac{1}{M}\left[\frac{2}{\gamma+1}\left(1+\frac{\gamma-1}{2}M^2\right)\right]^{\frac{\gamma+1}{2(\gamma-1)}}$$

input	meaning	condition
`ar`	area ratio A/A* (>= 1)	
`regime`	which branch: subsonic or supersonic	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M`	Mach number
`p_p0`	static-to-stagnation pressure ratio p/p_0
`T_T0`	static-to-stagnation temperature ratio T/T_0

- **Method:** solve A/A* = $(1/M)$[(2/(g+1))(1 + (g-1)/2 M^2)]^((g+1)/(2(g-1))) on the chosen branch by bisection
- **Assumes:** $1 \leq A/A^*$ <= 10000; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_mach_from_area_ratio', ar=1.6875, regime='supersonic')
# -> M = 2, p_p0 = 0.127805, T_T0 = 0.555556   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_mach_from_area_ratio', ar=1.0, regime='subsonic')
# -> M = 1, p_p0 = 0.528282, T_T0 = 0.833333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mach from area ratio: ar = 1.6875, regime = supersonic	SOLVED	M = 2, p_p0 = 0.127805, T_T0 = 0.555556
Mach from area ratio: A/A* = 1, regime = subsonic	SOLVED	M = 1, p_p0 = 0.528282, T_T0 = 0.833333
Mach from area ratio: ar = 0.5, regime = subsonic	INVALID_INPUT	Mach number from the area ratio A/A*: the area ratio A/A* must lie between 1 and 10000.
Mach from area ratio: ar = 2	NEEDS_INPUT	aero_mach_from_area_ratio needs regime (which branch: subsonic or supersonic). Example: solve('aero_mach_from_area_ratio

Mach number from the stagnation-to-static pressure ratio — `aero_mach_from_pressure_ratio`

The Mach number of isentropic flow from the ratio of stagnation to static pressure p_0/p, with T_0/T and A/A.

$$M = \sqrt{\frac{2}{\gamma - 1}\left[\left(\frac{p_0}{p}\right)^{\frac{\gamma-1}{\gamma}} - 1\right]}$$

input	meaning	condition
`pr`	stagnation-to-static pressure ratio p_0/p (> 1)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M`	Mach number
`T0_T`	stagnation-to-static temperature ratio
`A_Astar`	area ratio A/A*

- **Method:** $M = sqrt(2/(g-1)$ [(p_0/p)^((g-1)/g) - 1])
- **Assumes:** $1 < p_0/p \leq 1e6$; $M \leq 20$; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_mach_from_pressure_ratio', pr=7.824449066867265)
# -> M = 2, T0_T = 1.8, A_Astar = 1.6875   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_mach_from_pressure_ratio', pr=1.8929291587378538)
# -> M = 1, T0_T = 1.2, A_Astar = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mach from pressure ratio: p0/p = 7.824449066867265	SOLVED	M = 2, T0_T = 1.8, A_Astar = 1.6875
Mach from pressure ratio: pr = 1.8929291587378538, gamma = 1.4	SOLVED	M = 1, T0_T = 1.2, A_Astar = 1
Mach from pressure ratio: p0/p = 0.5	INVALID_INPUT	Mach number from the stagnation-to-static pressure ratio: the pressure ratio p0/p must lie between 1 and 1e6.
Mach from pressure ratio: gamma = 1.4	NEEDS_INPUT	aero_mach_from_pressure_ratio needs pr (stagnation-to-static pressure ratio p0/p (> 1)). Example: solve('aero_mach_from_

Normal shock relations — `aero_normal_shock`

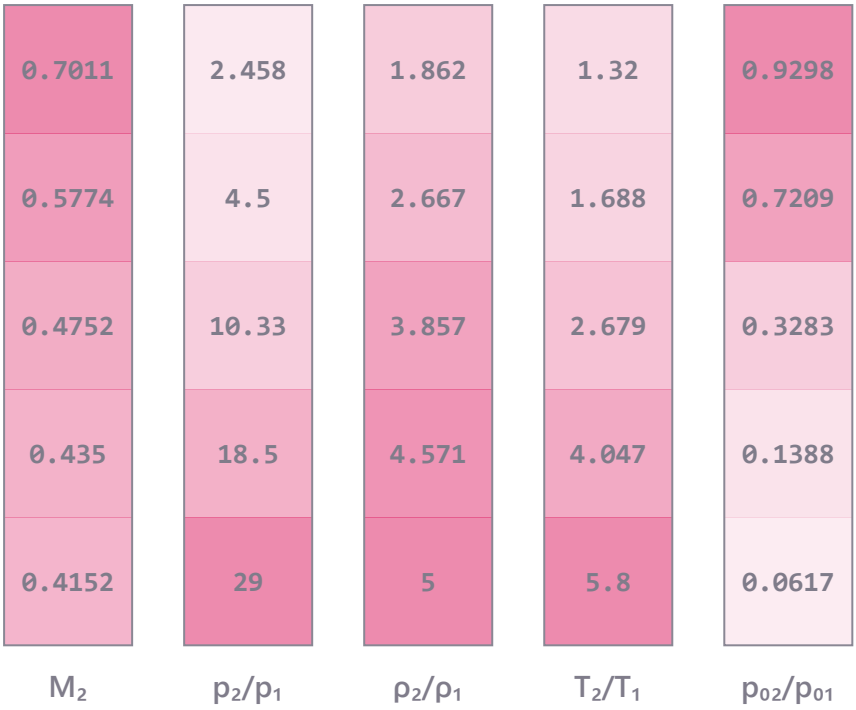
Downstream Mach number and the pressure, temperature, density and stagnation-pressure ratios across a normal shock, with the pitot ratio $p02/p_1$.

$$\frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma+1}(M_1^2 - 1), \quad \frac{\rho_2}{\rho_1} = \frac{(\gamma+1)M_1^2}{(\gamma-1)M_1^2 + 2}, \quad M_2^2 = \frac{1 + \frac{\gamma-1}{2}M_1^2}{\gamma M_1^2 - \frac{\gamma-1}{2}}$$

input	meaning	condition
`M1`	upstream Mach number (> 1)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M2`	downstream Mach number
`p2_p1`	static pressure ratio p_2/p_1
`T2_T1`	temperature ratio T_2/T_1
`rho2_rho1`	density ratio $rho2/rho1$
`p02_p01`	stagnation pressure ratio $p02/p01$
`p02_p1`	pitot ratio $p02/p_1$

- **Method:** $p_2/p_1 = 1 + 2g/(g+1)(M_1^2 - 1)$; rho2/rho1 = (g+1)$M_1^2$/((g-1)M1^2 + 2); M2^2 = (1 + (g-1)/2 M_1^2)/(g M1^2 - (g-1)/2)
- **Assumes:** $1 < M_1 \leq 20$; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The normal-shock table for air (γ = 1.4): for upstream Mach numbers M_1 = 1.5, 2, 3, 4, 5 (rows, top to bottom) the downstream Mach number and the pressure, density, temperature and stagnation-pressure ratios, every cell computed by the package and each column shaded by its own largest value; at M_1 = 2: M_2 = 0.57735, p_2/p_1 = 4.5, ρ_2/ρ_1 = 2.66667, T_2/T_1 = 1.6875, p_{02}/p_{01} = 0.720874.

```
aerospace_engineering_ai999x.solve('aero_normal_shock', M1=2.0)
# -> M2 = 0.57735, p2_p1 = 4.5, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 = 0.720874, p02_p1 =
5.64044   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_normal_shock', M1=3.0)
# -> M2 = 0.475191, p2_p1 = 10.3333, T2_T1 = 2.67901, rho2_rho1 = 3.85714, p02_p01 = 0.328344,
p02_p1 = 12.061   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal shock: M1 = 2	SOLVED	M2 = 0.57735, p2_p1 = 4.5, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 = 0.720874, p02_p1 = 5.64044
Normal shock: M1 = 2, gamma = 1.4	SOLVED	M2 = 0.57735, p2_p1 = 4.5, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 = 0.720874, p02_p1 = 5.64044
Normal shock: M1 = 0.8	INVALID_INPUT	Normal shock relations: the upstream Mach number must lie between 1 and 20.
Normal shock: gamma = 1.4	NEEDS_INPUT	aero_normal_shock needs M1 (upstream Mach number (> 1)). Example: solve('aero_normal_shock', M1=2).

Oblique shock from the deflection angle — `aero_oblique_shock`

The wave angle, downstream Mach number and the pressure, temperature, density and stagnation-pressure ratios of an attached oblique shock turning a supersonic flow by a deflection angle (weak or strong solution).

$$\tan\theta = 2\cot\beta\,\frac{M_1^2\sin^2\beta - 1}{M_1^2(\gamma + \cos 2\beta) + 2}, \quad M_{n1} = M_1\sin\beta, \quad M_2 = \frac{M_{n2}}{\sin(\beta - \theta)}$$

input	meaning	condition
`M1`	upstream Mach number (> 1)	
`theta`	flow deflection (wedge half-) angle (deg, rad)	
`branch`	weak (default) or strong shock solution	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`beta_deg`	shock wave angle in degrees
`M2`	downstream Mach number
`p2_p1`	pressure ratio p_2/p_1
`T2_T1`	temperature ratio T_2/T_1
`rho2_rho1`	density ratio $rho2/rho1$
`p02_p01`	stagnation pressure ratio
`Mn1`	upstream Mach number normal to the shock

- **Method:** theta-beta-M relation tan(theta) = 2 cot(beta) (M1^2 $sin^2\,\beta - 1)/(M_1^2$ (g + cos 2 beta) + 2), solved by bisection on the chosen branch; normal-shock relations on $Mn1 = M_1$ sin beta; $M_2 = \frac{Mn_2}{\sin(\beta-\theta)}$
- **Assumes:** $1 < M_1 \leq 20$; 0 < theta < theta_max(M1) (attached shock); $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_oblique_shock', M1=2.0, theta=0.18540474909716562)
# -> beta_deg = 40, M2 = 1.61732, p2_p1 = 1.76149, T2_T1 = 1.18177, rho2_rho1 = 1.49056, p02_p01 =
0.981791, Mn1 = 1.28558   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_oblique_shock', M1=3.0, theta=0.4176000425542302,
branch='strong')
# -> beta_deg = 80, M2 = 0.576006, p2_p1 = 10.0167, T2_T1 = 2.62576, rho2_rho1 = 3.81478, p02_p01 =
0.341453, Mn1 = 2.95442   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Oblique shock: M1 = 2, theta = 10.62290962494955 deg	SOLVED	beta_deg = 40, M2 = 1.61732, p2_p1 = 1.76149, T2_T1 = 1.18177, rho2_rho1 = 1.49056, p02_p01 = 0.981791, Mn1 = 1.28558
Oblique shock: M1 = 3, theta = 23.92671996284097 deg, branch = strong	SOLVED	beta_deg = 80, M2 = 0.576006, p2_p1 = 10.0167, T2_T1 = 2.62576, rho2_rho1 = 3.81478, p02_p01 = 0.341453, Mn1 = 2.95442
Oblique shock: M1 = 2, theta = 30 deg	INVALID_INPUT	Oblique shock from the deflection angle: the deflection exceeds the maximum for an attached oblique shock (the shock det

question	outcome	result
Oblique shock: M1 = 2	NEEDS_INPUT	aero_oblique_shock needs theta (flow deflection (wedge half-) angle (deg, rad)). Example: solve('aero_oblique_shock', M1

Prandtl-Meyer angle of a Mach number — `aero_prandtl_meyer`

The Prandtl-Meyer function nu(M): the angle through which a sonic flow must turn isentropically to reach Mach M.

$$\nu(M) = \sqrt{\frac{\gamma+1}{\gamma-1}}\,\arctan\sqrt{\frac{\gamma-1}{\gamma+1}(M^2-1)} - \arctan\sqrt{M^2-1}$$

input	meaning	condition
`M`	Mach number (>= 1)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`nu_deg`	Prandtl-Meyer angle in degrees

- **Method:** $\nu = sqrt((g+1)/(g-1))$ atan(sqrt($(g-1)/(g+1)(M^2-1)$)) - atan(sqrt(M^2 - 1))
- **Assumes:** $1 \le M \le 50$; $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_prandtl_meyer', M=1.0)
# -> nu_deg = 0   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_prandtl_meyer', M=2.0)
# -> nu_deg = 26.3798   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Prandtl-Meyer angle: M = 2	SOLVED	nu_deg = 26.3798
Prandtl-Meyer angle: M = 1	SOLVED	nu_deg = 0
Prandtl-Meyer angle: M = 0.5	INVALID_INPUT	Prandtl-Meyer angle of a Mach number: the Mach number must lie between 1 and 50.
Prandtl-Meyer angle:	NEEDS_INPUT	aero_prandtl_meyer needs M (Mach number (>= 1)). Example: solve('aero_prandtl_meyer', M=1).

Mach number from the Prandtl-Meyer angle — `aero_mach_from_prandtl_meyer`

The supersonic Mach number whose Prandtl-Meyer angle is nu, with the maximum turning angle of the gas (expansion to vacuum).

$$\nu(M) = \nu, \quad \nu_{max} = \frac{\pi}{2}\left(\sqrt{\frac{\gamma+1}{\gamma-1}} - 1\right)$$

input	meaning	condition
`nu`	Prandtl-Meyer angle (deg, rad)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M`	Mach number
`nu_max_deg`	maximum Prandtl-Meyer angle (M -> infinity) in degrees

- **Method:** invert nu(M) by bisection; nu_max = (sqrt($(g+1)/(g-1)$) - 1) $pi/2$
- **Assumes:** $0 \le \nu < \nu$(M = 50); $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_mach_from_prandtl_meyer', nu=0.46041368208269473)
# -> M = 2, nu_max_deg = 130.454   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_mach_from_prandtl_meyer', nu=0.0)
# -> M = 1, nu_max_deg = 130.454   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Mach from Prandtl-Meyer angle: nu = 26.379760813416457 deg	SOLVED	M = 2, nu_max_deg = 130.454
Mach from Prandtl-Meyer angle: nu = 0 deg	SOLVED	M = 1, nu_max_deg = 130.454
Mach from Prandtl-Meyer angle: nu = 150 deg	INVALID_INPUT	Mach number from the Prandtl-Meyer angle: the Prandtl-Meyer angle must lie between 0 and nu(50).
Mach from Prandtl-Meyer angle: gamma = 1.4	NEEDS_INPUT	aero_mach_from_prandtl_meyer needs nu (Prandtl-Meyer angle (deg, rad)). Example: solve('aero_mach_from_prandtl_meyer', n

Prandtl-Meyer expansion fan — `aero_expansion_fan`

The downstream Mach number and the pressure, temperature and density ratios when a supersonic flow turns away from itself by an angle through a centred expansion fan.

$$\nu(M_2) = \nu(M_1) + \theta, \quad \frac{p_2}{p_1} = \left[\frac{1 + \frac{\gamma-1}{2}M_1^2}{1 + \frac{\gamma-1}{2}M_2^2}\right]^{\frac{\gamma}{\gamma-1}}$$

input	meaning	condition
`M1`	upstream Mach number (>= 1)	
`theta`	turning angle of the flow (deg, rad)	

input	meaning	condition
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M2`	downstream Mach number
`nu1_deg`	upstream Prandtl-Meyer angle (deg)
`nu2_deg`	downstream Prandtl-Meyer angle (deg)
`p2_p1`	pressure ratio p_2/p_1
`T2_T1`	temperature ratio T_2/T_1
`rho2_rho1`	density ratio $rho2/rho1$

- **Method:** $nu2 = \nu$(M1) + theta; M2 from nu(M2) = nu2; p_2/p_1 = [(1 + (g-1)/2 M_1^2)/(1 + $(g-1)/2$ M2^2)]^(g/(g-1)) (isentropic)
- **Assumes:** $1 \le M_1 \le 20$; $0 < \theta < 90$ deg; downstream Mach below 50; $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_expansion_fan', M1=1.0, theta=0.46041368208269473)
# -> M2 = 2, nu1_deg = 0, nu2_deg = 26.3798, p2_p1 = 0.241925, T2_T1 = 0.666667, rho2_rho1 =
0.362887   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Expansion fan: M1 = 1, theta = 26.379760813416457 deg	SOLVED	M2 = 2, nu1_deg = 0, nu2_deg = 26.3798, p2_p1 = 0.241925, T2_T1 = 0.666667, rho2_rho1 = 0.362887
Expansion fan: M1 = 0.5, theta = 10 deg	INVALID_INPUT	Prandtl-Meyer expansion fan: the upstream Mach number must lie between 1 and 20.
Expansion fan: M1 = 2	NEEDS_INPUT	aero_expansion_fan needs theta (turning angle of the flow (deg, rad)). Example: solve('aero_expansion_fan', M1=1, theta=

Normal shock from its pressure ratio — `aero_normal_shock_from_pressure_ratio`

The upstream and downstream Mach numbers and the temperature, density and stagnation-pressure ratios of a normal shock of known static pressure ratio p_2/p_1 (shock strength).

$$M_1 = \sqrt{\frac{\gamma+1}{2\gamma}\left(\frac{p_2}{p_1}-1\right)+1}$$

input	meaning	condition
`p21`	static pressure ratio across the shock p_2/p_1 (> 1)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`M1`	upstream Mach number
`M2`	downstream Mach number

output	meaning
`T2_T1`	temperature ratio
`rho2_rho1`	density ratio
`p02_p01`	stagnation pressure ratio

- **Method:** $M_1 = sqrt((g+1)/(2g))$ (p2/p1 - 1) + 1), then the normal-shock relations
- **Assumes:** $1 < p_2/p_1 \leq 500$; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_normal_shock_from_pressure_ratio', p21=4.5)
# -> M1 = 2, M2 = 0.57735, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 = 0.720874   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_normal_shock_from_pressure_ratio', p21=10.333333333333334)
# -> M1 = 3, M2 = 0.475191, T2_T1 = 2.67901, rho2_rho1 = 3.85714, p02_p01 = 0.328344   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal shock from pressure ratio: p2/p1 = 4.5	SOLVED	M1 = 2, M2 = 0.57735, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 = 0.720874
Normal shock from pressure ratio: p2/p1 = 0.5	INVALID_INPUT	Normal shock from its pressure ratio: the static pressure ratio p2/p1 must lie between 1 and 500.
Normal shock from pressure ratio: gamma = 1.4	NEEDS_INPUT	aero_normal_shock_from_pressure_ratio needs p21 (static pressure ratio across the shock p2/p1 (> 1)). Example: solve('ae

Oblique shock from the wave angle — `aero_oblique_shock_from_wave_angle`

The flow deflection, downstream Mach number and the property ratios of an oblique shock of known wave angle (for example measured on a schlieren photograph).

$$\tan\theta = 2\cot\beta\,\frac{M_1^2\sin^2\beta - 1}{M_1^2(\gamma + \cos 2\beta) + 2}$$

input	meaning	condition
`M1`	upstream Mach number (> 1)	
`beta`	shock wave angle (deg, rad)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`theta_deg`	flow deflection angle in degrees
`M2`	downstream Mach number
`p2_p1`	pressure ratio
`T2_T1`	temperature ratio
`rho2_rho1`	density ratio

output	meaning
`p02_p01`	stagnation pressure ratio
`Mn1`	upstream normal Mach number

- **Method:** theta from the theta-beta-M relation; normal-shock relations on $Mn1 = M_1$ sin beta; $M_2 = \frac{Mn_2}{\sin(\beta-\theta)}$
- **Assumes:** $1 < M_1 \leq 20$; asin(1/M1) < beta <= 90 deg; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_oblique_shock_from_wave_angle', M1=2.0,
beta=1.5707963267948966)
# -> theta_deg = 0, M2 = 0.57735, p2_p1 = 4.5, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 =
0.720874, Mn1 = 2   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_oblique_shock_from_wave_angle', M1=2.0,
beta=0.6981317007977318)
# -> theta_deg = 10.6229, M2 = 1.61732, p2_p1 = 1.76149, T2_T1 = 1.18177, rho2_rho1 = 1.49056,
p02_p01 = 0.981791, Mn1 = 1.28558   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Oblique shock from wave angle: M1 = 2, beta = 90 deg	SOLVED	theta_deg = 0, M2 = 0.57735, p2_p1 = 4.5, T2_T1 = 1.6875, rho2_rho1 = 2.66667, p02_p01 = 0.720874, Mn1 = 2
Oblique shock from wave angle: M1 = 2, beta = 20 deg	INVALID_INPUT	Oblique shock from the wave angle: the wave angle must lie between the Mach angle asin(1/M1) and 90 degrees.
Oblique shock from wave angle: M1 = 2	NEEDS_INPUT	aero_oblique_shock_from_wave_angle needs beta (shock wave angle (deg, rad)). Example: solve('aero_oblique_shock_from_wav

Maximum deflection angle of an attached oblique shock — `aero_max_deflection_angle`

The largest wedge (deflection) angle for which an oblique shock stays attached at a given upstream Mach number, with the wave angle, downstream Mach number and pressure ratio at that limit; beyond it the shock detaches.

$$\sin^2\beta^* = \frac{\frac{\gamma+1}{4}M_1^2 - 1 + \sqrt{(\gamma+1)\left(1 + \frac{\gamma-1}{2}M_1^2 + \frac{\gamma+1}{16}M_1^4\right)}}{\gamma M_1^2}$$

input	meaning	condition
`M1`	upstream Mach number (>= 1.01)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`theta_max_deg`	maximum deflection angle in degrees
`beta_deg`	wave angle at the maximum, degrees

output	meaning
`M2`	downstream Mach number at the maximum
`p2_p1`	pressure ratio at the maximum

- **Method:** sin^2 beta* = [(g+1)$M^2/4 - 1 + sqrt((g + 1)$(1 + (g-1)M^2/2 + (g+1)M^4/16))]/(g M^2); theta_max = theta(beta*)
- **Assumes:** $1.01 \le M_1 \le 20$; $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_max_deflection_angle', M1=2.0)
# -> theta_max_deg = 22.9735, beta_deg = 64.669, M2 = 0.924256, p2_p1 = 3.64575   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_max_deflection_angle', M1=3.0)
# -> theta_max_deg = 34.0734, beta_deg = 65.2408, M2 = 0.954047, p2_p1 = 8.49166   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum deflection angle: M1 = 2	SOLVED	theta_max_deg = 22.9735, beta_deg = 64.669, M2 = 0.924256, p2_p1 = 3.64575
Maximum deflection angle: M1 = 0.9	INVALID_INPUT	Maximum deflection angle of an attached oblique shock: the upstream Mach number must lie between 1.01 and 20.
Maximum deflection angle: gamma = 1.4	NEEDS_INPUT	aero_max_deflection_angle needs M1 (upstream Mach number (>= 1.01)). Example: solve('aero_max_deflection_angle', M1=2).

Fanno flow (adiabatic duct flow with friction) — `aero_fanno_flow`

For adiabatic flow with wall friction in a constant-area duct: the friction length f L*/D needed to reach Mach 1 from Mach M, and the pressure, temperature, density and stagnation-pressure ratios to the sonic (choked) state.

$$\frac{\bar{f}L^*}{D} = \frac{1-M^2}{\gamma M^2} + \frac{\gamma+1}{2\gamma}\ln\frac{(\gamma+1)M^2}{2+(\gamma-1)M^2}, \quad \frac{p}{p^*} = \frac{1}{M}\sqrt{\frac{\gamma+1}{2+(\gamma-1)M^2}}$$

input	meaning	condition
`M`	Mach number at the duct station (0.05 to 10, not 1)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`fLD`	f L*/D with the Darcy friction factor f (= 4 times the Fanning factor)
`p_pstar`	p/p*
`T_Tstar`	T/T*
`rho_rhostar`	rho/rho* (= V*/V)
`p0_p0star`	$p_0/p_0{}^*$

- **Method:** f L*/D = (1 - M^2)/(g M^2) + (g+1)/(2g) ln[(g+1)M^2/(2 + (g-1)M^2)]; T/T* = (g+1)/(2 + (g-1)M^2); p/p* = sqrt(T/T*)/M; p_0/p_0* = (T*/T)^((g+1)/(2(g-1)))/M
- **Assumes:** $0.05 \le M \le 10$; adiabatic, constant area, constant friction factor; $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_fanno_flow', M=2.0)
# -> fLD = 0.304997, p_pstar = 0.408248, T_Tstar = 0.666667, rho_rhostar = 0.612372, p0_p0star =
1.6875   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_fanno_flow', M=0.5)
# -> fLD = 1.06906, p_pstar = 2.13809, T_Tstar = 1.14286, rho_rhostar = 1.87083, p0_p0star =
1.33984   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fanno flow: M = 2	SOLVED	fLD = 0.304997, p_pstar = 0.408248, T_Tstar = 0.666667, rho_rhostar = 0.612372, p0_p0star = 1.6875
Fanno flow: M = 1	INVALID_INPUT	Fanno flow (adiabatic duct flow with friction): the Mach number must lie between 0.05 and 10 (not 1).
Fanno flow: gamma = 1.4	NEEDS_INPUT	aero_fanno_flow needs M (Mach number at the duct station (0.05 to 10, not 1)). Example: solve('aero_fanno_flow', M=2).

Rayleigh flow (duct flow with heat addition) — `aero_rayleigh_flow`

For frictionless constant-area flow with heat addition: the stagnation temperature, temperature, pressure, density and stagnation-pressure ratios to the sonic (thermally choked) state at a Mach number.

$$\frac{p}{p^*} = \frac{1+\gamma}{1+\gamma M^2}, \quad \frac{T_0}{T_0^*} = \frac{(\gamma+1)M^2\left[2+(\gamma-1)M^2\right]}{(1+\gamma M^2)^2}$$

input	meaning	condition
`M`	Mach number (0.05 to 10)	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`T0_T0star`	T_0/T_0*
`T_Tstar`	T/T*
`p_pstar`	p/p*
`rho_rhostar`	rho/rho*
`p0_p0star`	p_0/p_0*

- **Method:** p/p* = (1+g)/(1 + g M^2); T/T* = M^2 (p/p*)^2; T_0/T_0* = (g+1)M^2(2 + (g-1)M^2)/(1 + g M^2)^2; p_0/p_0* = (p/p*)[(2 + (g-1)M^2)/(g+1)]^(g/(g-1))
- **Assumes:** $0.05 \le M \le 10$; frictionless, constant area; $1.05 \le \gamma \le 1.7$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_rayleigh_flow', M=1.0)
# -> T0_T0star = 1, T_Tstar = 1, p_pstar = 1, rho_rhostar = 1, p0_p0star = 1   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_rayleigh_flow', M=2.0)
# -> T0_T0star = 0.793388, T_Tstar = 0.528926, p_pstar = 0.363636, rho_rhostar = 0.6875, p0_p0star
= 1.5031   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rayleigh flow: M = 2	SOLVED	T0_T0star = 0.793388, T_Tstar = 0.528926, p_pstar = 0.363636, rho_rhostar = 0.6875, p0_p0star = 1.5031
Rayleigh flow: M = 20	INVALID_INPUT	Rayleigh flow (duct flow with heat addition): the Mach number must lie between 0.05 and 10.
Rayleigh flow:	NEEDS_INPUT	aero_rayleigh_flow needs M (Mach number (0.05 to 10)). Example: solve('aero_rayleigh_flow', M=1).

Choked mass flow through a nozzle throat — `aero_choked_mass_flow`

The maximum (choked) mass flow of a perfect gas through a sonic throat from the stagnation pressure and temperature, with the throat (critical) pressure, temperature and speed.

$$\dot{m} = \frac{p_0 A^*}{\sqrt{R T_0}} \sqrt{\gamma} \left(\frac{2}{\gamma+1} \right)^{\frac{\gamma+1}{2(\gamma-1)}}$$

input	meaning	condition
`p0`	stagnation (reservoir) pressure	
`T0`	stagnation temperature	
`A`	throat area (m^2, cm^2, mm^2)	
`gamma`	ratio of specific heats (default 1.4, air)	
`R`	specific gas constant (default air, 287.053 $J/(kgK)$)	

output	meaning
`mdot`	choked mass flow rate
`p_star`	throat pressure
`T_star`	throat temperature
`V_star`	throat (sonic) speed

- **Method:** mdot = Gamma p0 A/sqrt(R T0), $Gamma = sqrt$(g)(2/(g+1))^((g+1)/(2(g-1))); p*/p0 = (2/(g+1))^(g/(g-1)); T*/$T_0 = 2/(g+1)$
- **Assumes:** p0, T0, A, $R > 0$; isentropic flow to a sonic throat; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_choked_mass_flow', p0=100000.0, T0=300.0, A=0.001)
# -> mdot = 0.233334 kg/s, p_star = 52828.2 Pa, T_star = 250 K, V_star = 316.968 m/s   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_choked_mass_flow', p0=100000.0, T0=300.0, A=0.001,
gamma=1.4, R=300.0)
# -> mdot = 0.228244 kg/s, p_star = 52828.2 Pa, T_star = 250 K, V_star = 324.037 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Choked mass flow: p0 = 100 kPa, T0 = 300 K, A = 10 cm^2	SOLVED	mdot = 0.233334 kg/s, p_star = 52828.2 Pa, T_star = 250 K, V_star = 316.968 m/s
Choked mass flow: p0 = 100 kPa, T0 = 300 K, A = 0 cm^2	INVALID_INPUT	Choked mass flow through a nozzle throat: the stagnation pressure, stagnation temperature, throat area and gas constant
Choked mass flow: p0 = 100 kPa, T0 = 300 K	NEEDS_INPUT	aero_choked_mass_flow needs A (throat area (m^2, cm^2, mm^2)). Example: solve('aero_choked_mass_flow', p0='100000 Pa', T

Characteristic velocity c* of a rocket propellant — `aero_characteristic_velocity`

The characteristic exhaust velocity c* = pc At / mdot of ideal rocket combustion products from the chamber temperature, molar mass and ratio of specific heats, with the Vandenkerckhove function.

$$c^* = \frac{\sqrt{RT_c}}{\Gamma}, \quad \Gamma = \sqrt{\gamma}\left(\frac{2}{\gamma+1}\right)^{\frac{\gamma+1}{2(\gamma-1)}}$$

input	meaning	condition
`Tc`	chamber (combustion) temperature	
`Mw`	molar mass of the combustion products (g/mol, kg/kmol)	
`gamma`	ratio of specific heats of the products	

output	meaning
`c_star`	characteristic velocity c*
`Gamma`	Vandenkerckhove function Gamma(gamma)

- **Method:** c* = sqrt(R Tc)/Gamma, $R = Ru/Mw$, $Gamma = sqrt$(g)(2/(g+1))^((g+1)/(2(g-1)))
- **Assumes:** $Tc > 0$; $1 \le Mw \le 300$ g/mol; $1.05 \le \gamma \le 1.7$; ideal, frozen, isentropic flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_characteristic_velocity', Tc=3000.0, Mw=0.02, gamma=1.2)
# -> c_star = 1721.99 m/s, Gamma = 0.648531   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Characteristic velocity: Tc = 3000 K, Mw = 20 g/mol, gamma = 1.2	SOLVED	c_star = 1721.99 m/s, Gamma = 0.648531

question	outcome	result
Characteristic velocity: Tc = 3000 K, Mw = 20 g/mol, gamma = 2.5	INVALID_INPUT	Characteristic velocity c* of a rocket propellant: the ratio of specific heats gamma must lie between 1.05 and 1.7.
Characteristic velocity: Tc = 3000 K, Mw = 20 g/mol	NEEDS_INPUT	aero_characteristic_velocity needs gamma (ratio of specific heats of the products). Example: solve('aero_characteristic_

Nozzle thrust coefficient — `aero_thrust_coefficient`

The thrust coefficient CF = F/(pc At) of an ideal rocket nozzle of given expansion area ratio at an ambient pressure, with the exit pressure ratio and Mach number and the vacuum thrust coefficient.

$$C_F = \sqrt{\frac{2\gamma^2}{\gamma-1}\left(\frac{2}{\gamma+1}\right)^{\frac{\gamma+1}{\gamma-1}}\left[1-\left(\frac{p_e}{p_c}\right)^{\frac{\gamma-1}{\gamma}}\right]} + \frac{p_e - p_a}{p_c}\frac{A_e}{A_t}$$

input	meaning	condition
`gamma`	ratio of specific heats of the exhaust	
`eps`	nozzle expansion area ratio Ae/At (> 1)	
`pc`	chamber pressure	
`pa`	ambient pressure (0 for vacuum)	

output	meaning
`CF`	thrust coefficient
`pe_pc`	exit-to-chamber pressure ratio
`Me`	exit Mach number
`CF_vac`	vacuum thrust coefficient

- **Method:** Me from the area ratio (supersonic); pe/pc isentropic; $CF = Gamma$ sqrt($2g/(g-1)$(1 - (pe/pc)^((g-1)/g))) + (pe - pa)/pc eps
- **Assumes:** 1 < eps <= 500; $0 \le pa < pc$; $pe \ge 0.4$ pa (no flow separation); $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_thrust_coefficient', gamma=1.4, eps=1.6875, pc=1000000.0,
pa=0.0)
# -> CF = 1.42342, pe_pc = 0.127805, Me = 2, CF_vac = 1.42342   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thrust coefficient: gamma = 1.4, eps = 1.6875, pc = 1 MPa, pa = 0 Pa	SOLVED	CF = 1.42342, pe_pc = 0.127805, Me = 2, CF_vac = 1.42342
Thrust coefficient: gamma = 1.2, eps = 40, pc = 1 MPa, pa = 101325 Pa	INVALID_INPUT	Nozzle thrust coefficient: the nozzle is so overexpanded (pe < 0.4 pa) that the flow is likely to separate; ideal thrust
Thrust coefficient: gamma = 1.2, eps = 40, pc = 7 MPa	NEEDS_INPUT	aero_thrust_coefficient needs pa (ambient pressure (0 for vacuum)). Example: solve('aero_thrust_coefficient', gamma=1.4,

Ideal rocket exhaust velocity — `aero_ideal_exhaust_velocity`

The exhaust velocity of an ideal rocket nozzle expanding combustion products from the chamber to the exit pressure, with the specific impulse at that exit pressure (optimum expansion) and the exit Mach number.

$$v_e = \sqrt{\frac{2\gamma}{\gamma-1}\frac{R_u T_c}{\mathcal{M}}\left[1-\left(\frac{p_e}{p_c}\right)^{\frac{\gamma-1}{\gamma}}\right]}$$

input	meaning	condition
`Tc`	chamber temperature	
`Mw`	molar mass of the products (g/mol)	
`gamma`	ratio of specific heats	
`pc`	chamber pressure	
`pe`	nozzle exit pressure (> 0)	

output	meaning
`ve`	exhaust velocity
`Isp`	specific impulse ve/g_0 (exit pressure = ambient)
`Me`	exit Mach number

- **Method:** $ve = sqrt(2g/(g-1)$ (Ru/Mw) Tc [$1-(pe/pc)$^((g-1)/g)])
- **Assumes:** $0 < pe < pc$; $1 \le Mw \le 300$ g/mol; $1.05 \le \gamma \le 1.7$; ideal, frozen, isentropic expansion
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_ideal_exhaust_velocity', Tc=3000.0, Mw=0.02, gamma=1.4,
pc=1000000.0, pe=127804.52546295093)
# -> ve = 1969.79 m/s, Isp = 200.863 s, Me = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exhaust velocity: Tc = 3000 K, Mw = 20 g/mol, gamma = 1.4, pc = 1 MPa, pe = 127.80452546295093 kPa	SOLVED	ve = 1969.79 m/s, Isp = 200.863 s, Me = 2
Exhaust velocity: Tc = 3000 K, Mw = 20 g/mol, gamma = 1.2, pc = 1 MPa, pe = 2 MPa	INVALID_INPUT	Ideal rocket exhaust velocity: the exit pressure must be at least 0 and below the chamber pressure.
Exhaust velocity: Tc = 3000 K, Mw = 20 g/mol, gamma = 1.2, pc = 1 MPa	NEEDS_INPUT	aero_ideal_exhaust_velocity needs pe (nozzle exit pressure (> 0)). Example: solve('aero_ideal_exhaust_velocity', Tc='300

Optimum nozzle area ratio — `aero_optimum_area_ratio`

The nozzle expansion area ratio that expands the exhaust exactly to the ambient pressure (optimum expansion, maximum thrust at that pressure), with the exit Mach number and the thrust coefficient.

$$M_e = \sqrt{\frac{2}{\gamma-1}\left[\left(\frac{p_c}{p_a}\right)^{\frac{\gamma-1}{\gamma}} - 1\right]}, \quad \varepsilon = \frac{A}{A^*}(M_e)$$

input	meaning	condition
`gamma`	ratio of specific heats	
`pc`	chamber pressure	
`pa`	ambient pressure	

output	meaning
`eps`	optimum area ratio Ae/At
`Me`	exit Mach number
`CF`	thrust coefficient at optimum expansion

- **Method:** $pe = pa$: $Me = sqrt(2/(g-1)$[(pc/pa)^((g-1)/g) - 1]); eps = A/A*(Me); $CF = Gamma$ sqrt($2g/(g-1)$(1 - (pa/pc)^((g-1)/g)))
- **Assumes:** $2 \le pc/pa \le 1e6$; $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_optimum_area_ratio', gamma=1.4, pc=782444.9066867265,
pa=100000.0)
# -> eps = 1.6875, Me = 2, CF = 1.20775   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Optimum area ratio: gamma = 1.4, pc = 782.4449066867265 kPa, pa = 100 kPa	SOLVED	eps = 1.6875, Me = 2, CF = 1.20775
Optimum area ratio: gamma = 1.2, pc = 100 kPa, pa = 100 kPa	INVALID_INPUT	Optimum nozzle area ratio: the chamber-to-ambient pressure ratio must lie between 2 and 1e6.
Optimum area ratio: gamma = 1.2, pc = 7 MPa	NEEDS_INPUT	aero_optimum_area_ratio needs pa (ambient pressure). Example: solve('aero_optimum_area_ratio', gamma=1.4, pc='782445 Pa'

Ideal rocket engine performance — `aero_rocket_engine_performance`

Thrust, mass flow, specific impulse, thrust coefficient, characteristic velocity and the exit pressure, velocity and Mach number of an ideal rocket engine from its chamber conditions, propellant products, throat area, expansion ratio and the ambient pressure.

$$F = \dot{m} v_e + (p_e - p_a)A_e, \quad \dot{m} = \frac{p_c A_t}{c^*}, \quad I_{sp} = \frac{F}{\dot{m} g_0}$$

input	meaning	condition
`pc`	chamber pressure	
`Tc`	chamber temperature	

input	meaning	condition
`Mw`	molar mass of the products (g/mol)	
`gamma`	ratio of specific heats	
`At`	throat area	
`eps`	expansion area ratio Ae/At	
`pa`	ambient pressure (0 for vacuum)	

output	meaning
`F`	thrust
`mdot`	propellant mass flow
`Isp`	specific impulse
`CF`	thrust coefficient
`c_star`	characteristic velocity
`pe`	exit pressure
`ve`	exit (exhaust) velocity
`Me`	exit Mach number

- **Method:** mdot = pc At/c*; Me from eps; $pe = pc$ (1 + (g-1)/2 $Me^2)^{(- g/(g - 1)}$); ve from the isentropic expansion; F = mdot ve + (pe - pa) Ae; Isp = F/(mdot g0)
- **Assumes:** 1 < eps <= 500; $0 \leq pa < pc$; $pe \geq 0.4$ pa (no flow separation); $1 \leq Mw \leq 300$ g/mol; $1.05 \leq \gamma \leq 1.7$; ideal, frozen, isentropic, one-dimensional nozzle flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_rocket_engine_performance', pc=1000000.0, Tc=3000.0,
Mw=0.02, gamma=1.4, At=0.01, eps=1.6875, pa=127804.52546295093)
# -> F = 12077.5 N, mdot = 6.13137 kg/s, Isp = 200.863 s, CF = 1.20775, c_star = 1630.96 m/s, pe =
127805 Pa, ve = 1969.79 m/s, Me = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rocket engine performance: pc = 1 MPa, Tc = 3000 K, Mw = 20 g/mol, gamma = 1.4, At = 0.01 m^2, eps = 1.6875, pa = 0 Pa	SOLVED	F = 14234.2 N, mdot = 6.13137 kg/s, Isp = 236.731 s, CF = 1.42342, c_star = 1630.96 m/s, pe = 127805 Pa, ve = 1969.79 m/s, Me = 2
Rocket engine performance: pc = 1 MPa, Tc = 3000 K, Mw = 20 g/mol, gamma = 1.2, At = 0.01 m^2, eps = 40, pa = 2 MPa	INVALID_INPUT	Ideal rocket engine performance: the ambient pressure must be at least 0 and below the chamber pressure.
Rocket engine performance: pc = 1 MPa, Tc = 3000 K, Mw = 20 g/mol, gamma = 1.2, eps = 40, pa = 0 Pa	NEEDS_INPUT	aero_rocket_engine_performance needs At (throat area). Example: solve('aero_rocket_engine_performance', pc='1e+06 Pa', T

Throat area of a rocket engine for a required thrust — `aero_throat_area_for_thrust`

The throat and exit areas, mass flow and specific impulse of an ideal rocket engine sized to give a required thrust at an ambient pressure.

$$A_t = \frac{F}{C_F\, p_c}, \quad \dot{m} = \frac{p_c A_t}{c^*}, \quad I_{sp} = \frac{c^* C_F}{g_0}$$

input	meaning	condition
`F`	required thrust (N, kN, lbf)	
`pc`	chamber pressure	
`Tc`	chamber temperature	
`Mw`	molar mass of the products	
`gamma`	ratio of specific heats	
`eps`	expansion area ratio Ae/At	
`pa`	ambient pressure (0 for vacuum)	

output	meaning
`At`	throat area
`Ae`	exit area
`mdot`	propellant mass flow
`Isp`	specific impulse

- **Method:** At = F/(CF pc); Ae = eps At; mdot = pc At/c*; Isp = c* CF/g0
- **Assumes:** $F > 0$; 1 < eps <= 500; $0 \le pa < pc$; $pe \ge 0.4$ pa; $1.05 \le \gamma \le 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_throat_area_for_thrust', F=1000.0, pc=1000000.0,
Tc=3000.0, Mw=0.02, gamma=1.4, eps=1.6875, pa=127804.52546295093)
# -> At = 0.000827984 m^2, Ae = 0.00139722 m^2, mdot = 0.507668 kg/s, Isp = 200.863 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Throat area for thrust: F = 1 kN, pc = 1 MPa, Tc = 3000 K, Mw = 20 g/mol, gamma = 1.4, eps = 1.6875, pa = 127.80452546295093 kPa	SOLVED	At = 0.000827984 m^2, Ae = 0.00139722 m^2, mdot = 0.507668 kg/s, Isp = 200.863 s
Throat area for thrust: F = -1 kN, pc = 1 MPa, Tc = 3000 K, Mw = 20 g/mol, gamma = 1.4, eps = 2, pa = 0 Pa	INVALID_INPUT	Throat area of a rocket engine for a required thrust: the thrust must be a positive number.
Throat area for thrust: F = 1 kN, pc = 1 MPa, Tc = 3000 K, Mw = 20 g/mol, gamma = 1.4, eps = 2	NEEDS_INPUT	aero_throat_area_for_thrust needs pa (ambient pressure (0 for vacuum)). Example: solve('aero_throat_area_for_thrust', F=

Normal shock position in a converging-diverging nozzle — `aero_nozzle_normal_shock`

Where a normal shock stands in the diverging part of a converging-diverging nozzle for a given back pressure: the area ratio at the shock, the Mach numbers before and after it, the exit Mach number and the stagnation-pressure loss.

$$\frac{p_e A_e}{p_{01} A_t} = \frac{p_e}{p_{02}} \frac{A_e}{A_2^*}\Big|_{M_e}, \quad \frac{p_{02}}{p_{01}} = \frac{A_t}{A_2^*}$$

input	meaning	condition
`pc`	reservoir (stagnation) pressure	
`pb`	back pressure at the exit	
`eps`	exit-to-throat area ratio Ae/At	
`gamma`	ratio of specific heats (default 1.4, air)	

output	meaning
`As_At`	area ratio at the shock As/At
`M1`	Mach number just before the shock
`M2`	Mach number just after the shock
`Me`	exit Mach number (subsonic)
`p02_p01`	stagnation pressure ratio across the shock

- **Method:** mass flow: pe Ae/(p01 At) = (p/p0)(A/A*) at Me (subsonic) gives Me; p02/p01 = pe/(p01 (p/p_0)(Me)); M1 from the normal-shock $p02/p01$; $As/At = A/A(M_1)$
- **Assumes:** 1.05 <= eps <= 100; back pressure between the exit-shock and the critical subsonic exit pressures; $1.05 \leq \gamma \leq 1.7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_nozzle_normal_shock', pc=1000000.0, pb=682558.5882398435,
eps=3.0)
# -> As_At = 1.6875, M1 = 2, M2 = 0.57735, Me = 0.280418, p02_p01 = 0.720874   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_nozzle_normal_shock', pc=1000000.0, pb=861358.2948493384,
eps=2.0)
# -> As_At = 1.17617, M1 = 1.5, M2 = 0.701089, Me = 0.332277, p02_p01 = 0.929787   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Normal shock in a nozzle: pc = 1 MPa, pb = 682.5585882398435 kPa, eps = 3	SOLVED	As_At = 1.6875, M1 = 2, M2 = 0.57735, Me = 0.280418, p02_p01 = 0.720874
Normal shock in a nozzle: pc = 1 MPa, pb = 999 kPa, eps = 3	INVALID_INPUT	Normal shock position in a converging-diverging nozzle: a normal shock stands inside the nozzle only for back pressures
Normal shock in a nozzle: pc = 1 MPa, eps = 3	NEEDS_INPUT	aero_nozzle_normal_shock needs pb (back pressure at the exit). Example: solve('aero_nozzle_normal_shock', pc='1e+06 Pa',

Critical Mach number of an airfoil — `aero_critical_mach`

The free-stream Mach number at which the flow first reaches sonic speed at the airfoil's minimum-pressure point, from its incompressible minimum pressure coefficient and a compressibility rule (Prandtl-Glauert or Karman-Tsien), air $(\gamma = 1.4)$.

$$C_{p,cr} = \frac{2}{\gamma M^2}\left[\left(\frac{2+(\gamma-1)M^2}{\gamma+1}\right)^{\frac{\gamma}{\gamma-1}} - 1\right] = \frac{C_{p0}}{\sqrt{1-M^2}}$$

input	meaning	condition
`Cp0`	incompressible minimum pressure coefficient (negative)	
`rule`	compressibility correction: prandtl-glauert or karman-tsien	

output	meaning
`M_cr`	critical Mach number
`Cp_cr`	critical (sonic) pressure coefficient at M_cr

- **Method:** Cp_cr(M) = 2/(g M^2) [((1 + (g-1)/2 M^2)/(1 + (g-1)/2))^(g/(g-1)) - 1]; equate to $Cp0/sqrt$ (1 - M^2) (or the Karman-Tsien form) and solve
- **Assumes:** $-3 \le Cp0 < -0.01$; subsonic, small-perturbation compressibility rules; air, $\gamma = 1.4$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_critical_mach', Cp0=-0.43, rule='prandtl-glauert')
# -> M_cr = 0.737106, Cp_cr = -0.636304   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Critical Mach: Cp0 = -0.43, rule = prandtl-glauert	SOLVED	M_cr = 0.737106, Cp_cr = -0.636304
Critical Mach: Cp0 = 0.5, rule = prandtl-glauert	INVALID_INPUT	Critical Mach number of an airfoil: the incompressible minimum pressure coefficient must lie between -3 and -0.01.
Critical Mach: Cp0 = -0.43	NEEDS_INPUT	aero_critical_mach needs rule (compressibility correction: prandtl-glauert or karman-tsien). Example: solve('aero_critic

Rocket delta-v from specific impulse (Tsiolkovsky) — `aero_rocket_delta_v`

The ideal velocity change of a rocket (no gravity or drag) from its specific impulse and its initial and burnout masses.

$$\Delta v = I_{sp}\, g_0 \ln \frac{m_0}{m_f}$$

input	meaning	condition
`Isp`	specific impulse (s)	
`m0`	initial (lift-off or ignition) mass	
`mf`	final (burnout) mass	

output	meaning
`dv`	ideal velocity change delta-v

- **Method:** dv = Isp g0 ln(m0/mf)

- **Assumes:** 1 s <= Isp <= 10000 s; $0 < mf < m_0$; no gravity, no drag
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_rocket_delta_v', Isp=300.0, m0=2718.2818284590453,
mf=1000.0)
# -> dv = 2941.99 m/s   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_rocket_delta_v', Isp=450.0, m0=4000.0, mf=1000.0)
# -> dv = 6117.71 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rocket delta-v: Isp = 450 s, m0 = 4000 kg, mf = 1000 kg	SOLVED	dv = 6117.71 m/s
Rocket delta-v: Isp = 300 s, m0 = 1000 kg, mf = 2000 kg	INVALID_INPUT	Rocket delta-v from specific impulse (Tsiolkovsky): the final (burnout) mass must be smaller than the initial mass.
Rocket delta-v: Isp = 300 s, m0 = 1000 kg	NEEDS_INPUT	aero_rocket_delta_v needs mf (final (burnout) mass). Example: solve('aero_rocket_delta_v', Isp='300 s', m0='2718.28 kg',

Propellant mass for a required delta-v — `aero_propellant_for_delta_v`

The propellant mass, initial mass and mass ratio a rocket stage needs to give a velocity change, from its specific impulse and burnout mass (structure plus payload).

$$\frac{m_0}{m_f} = e^{\Delta v/(I_{sp} g_0)}, \quad m_p = m_f \left(e^{\Delta v/(I_{sp} g_0)} - 1\right)$$

input	meaning	condition
`dv`	required velocity change (m/s, km/s)	
`Isp`	specific impulse (s)	
`mf`	burnout mass (dry mass plus payload)	

output	meaning
`m0`	initial mass
`mp`	propellant mass
`R`	mass ratio m_0/mf

- **Method:** $R = exp$(dv/(Isp g0)); $m_0 = R$ mf; $mp = (R - 1)$ mf
- **Assumes:** $dv > 0$; 1 s <= Isp <= 10000 s; $mf > 0$; mass ratio <= 10000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_propellant_for_delta_v', dv=2941.995, Isp=300.0,
mf=1000.0)
# -> m0 = 2718.28 kg, mp = 1718.28 kg, R = 2.71828   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Propellant for delta-v: dv = 2941.995 m/s, Isp = 300 s, mf = 1000 kg	SOLVED	m0 = 2718.28 kg, mp = 1718.28 kg, R = 2.71828
Propellant for delta-v: dv = -3 km/s, Isp = 300 s, mf = 1000 kg	INVALID_INPUT	Propellant mass for a required delta-v: the velocity change and the burnout mass must be a positive number.
Propellant for delta-v: dv = 3 km/s, Isp = 300 s	NEEDS_INPUT	aero_propellant_for_delta_v needs mf (burnout mass (dry mass plus payload)). Example: solve('aero_propellant_for_delta_v

Gravity loss of a vertical rocket burn — `aero_delta_v_gravity_loss`

The burnout speed of a rocket climbing vertically under constant thrust and constant gravity (no drag), its ideal delta-v and the gravity loss g tb.

$$v_b = I_{sp} g_0 \ln \frac{m_0}{m_f} - g\,t_b$$

input	meaning	condition
`Isp`	specific impulse	
`m0`	initial mass	
`mf`	burnout mass	
`tb`	burn time	
`g`	gravitational acceleration (default 9.80665 m/s^2)	

output	meaning
`v_b`	vertical speed at burnout
`dv_ideal`	ideal delta-v (no gravity)
`loss`	gravity loss

- **Method:** v_b = Isp g0 ln(m0/mf) - g tb
- **Assumes:** $0 < mf < m_0$; thrust-to-weight above 1 at lift-off; constant thrust and mass flow, constant g, no drag, vertical flight
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_delta_v_gravity_loss', Isp=300.0, m0=2718.2818284590453,
mf=1000.0, tb=100.0)
# -> v_b = 1961.33 m/s, dv_ideal = 2941.99 m/s, loss = 980.665 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gravity loss: Isp = 300 s, m0 = 4 t, mf = 1 t, tb = 100 s	SOLVED	v_b = 3097.81 m/s, dv_ideal = 4078.47 m/s, loss = 980.665 m/s
Gravity loss: Isp = 300 s, m0 = 4 t, mf = 1 t, tb = 1000 s	INVALID_INPUT	Gravity loss of a vertical rocket burn: the thrust must exceed the initial weight (thrust-to-weight above 1).

question	outcome	result
Gravity loss: Isp = 300 s, m0 = 4 t, mf = 1 t	NEEDS_INPUT	aero_delta_v_gravity_loss needs tb (burn time). Example: solve('aero_delta_v_gravity_loss', Isp='300 s', m0='2718.28 kg'

Burnout height and apogee of a vertical rocket — `aero_vertical_rocket_trajectory`

Burnout speed and height, apogee height and time to apogee of a rocket launched vertically under constant thrust and mass flow and constant gravity, neglecting drag.

$$h_b = c\,t_b\left(1 - \frac{\ln R}{R-1}\right) - \frac{1}{2}g t_b^2, \quad h_{max} = h_b + \frac{v_b^2}{2g}$$

input	meaning	condition
`Isp`	specific impulse	
`m0`	initial mass	
`mf`	burnout mass	
`tb`	burn time	
`g`	gravitational acceleration (default 9.80665 m/s^2)	

output	meaning
`v_b`	burnout speed
`h_b`	burnout height
`h_max`	apogee height
`t_apex`	time from lift-off to apogee

- **Method:** $v_b = c$ ln R - g tb; $h_b = c$ tb (1 - ln $R/(R-1)$) - g $tb^2/2$; h_max = h_b + v_b^2/(2g); t_apex = tb + v_b/g (c = Isp g0, R = m0/mf)
- **Assumes:** $0 < mf < m_0$; thrust-to-weight above 1; constant thrust, constant g, no drag; vertical flight
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_vertical_rocket_trajectory', Isp=300.0,
m0=2718.2818284590453, mf=1000.0, tb=100.0)
# -> v_b = 1961.33 m/s, h_b = 73949 m, h_max = 270082 m, t_apex = 300 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vertical rocket: Isp = 300 s, m0 = 2718.2818284590453 kg, mf = 1000 kg, tb = 100 s	SOLVED	v_b = 1961.33 m/s, h_b = 73949 m, h_max = 270082 m, t_apex = 300 s
Vertical rocket: Isp = 300 s, m0 = 1000 kg, mf = 400 kg, tb = 1000 s	INVALID_INPUT	Burnout height and apogee of a vertical rocket: the thrust must exceed the initial weight (thrust-to-weight above 1).
Vertical rocket: Isp = 300 s, m0 = 1000 kg, mf = 400 kg	NEEDS_INPUT	aero_vertical_rocket_trajectory needs tb (burn time). Example: solve('aero_vertical_rocket_trajectory', Isp='300 s', m0=

Delta-v of a multistage rocket — `aero_multistage_delta_v`

The total ideal velocity change of a multistage rocket and the contribution of each stage, from each stage's specific impulse and the vehicle mass at its ignition and at its burnout.

$$\Delta v = \sum_i I_{sp,i}\, g_0 \ln \frac{m_{0,i}}{m_{f,i}}$$

input	meaning	condition
`Isp`	specific impulse of each stage in seconds, e.g. '[290, 340]'	
`m0`	vehicle mass at each stage's ignition in kilograms	
`mf`	vehicle mass at each stage's burnout in kilograms	

output	meaning
`dv`	total delta-v
`dv_stages`	delta-v of each stage in m/s

- **Method:** dv = sum of Isp_i g0 ln(m0_i/mf_i)
- **Assumes:** equal numbers of stages; $0 < mf < m_0$ for each stage; each ignition mass below the previous burnout mass; no gravity or drag
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_multistage_delta_v', Isp='300, 300', m0=
[2718.2818284590453, 500.0], mf=[1000.0, 183.93972058572118])
# -> dv = 5883.99 m/s, dv_stages = [2941.995, 2941.995]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multistage rocket: Isp = [300, 300], m0 = [2718.2818284590453, 500], mf = [1000, 183.93972058572118]	SOLVED	dv = 5883.99 m/s, dv_stages = [2941.995, 2941.995]
Multistage rocket: Isp = [300, 300], m0 = [1000, 2000], mf = [500, 100]	INVALID_INPUT	Delta-v of a multistage rocket: stage 2: its ignition mass must be below the burnout mass of stage 1 (a stage is dropped
Multistage rocket: Isp = [300, 300], m0 = [1000, 400]	NEEDS_INPUT	aero_multistage_delta_v needs mf (vehicle mass at each stage's burnout in kilograms). Example: solve('aero_multistage_de

Payload fraction of a rocket with identical stages — `aero_identical_stages_payload`

For a rocket of n identical stages (same specific impulse and structural coefficient, equal delta-v split): the payload ratio of each stage, the overall payload fraction, the stage mass ratio and the gross lift-off mass per kilogram of payload.

$$R = e^{\Delta v/(n I_{sp} g_0)}, \quad \lambda = \frac{1-\varepsilon R}{R-1}, \quad \frac{m_{pl}}{m_0} = \left(\frac{\lambda}{1+\lambda}\right)^n$$

input	meaning	condition
`n`	number of stages (1 to 6)	
`dv`	total delta-v	
`Isp`	specific impulse of every stage	
`eps`	structural coefficient ms/(ms + mp) of every stage	

output	meaning
`lam`	payload ratio of each stage (payload above / stage mass)
`payload_fraction`	overall payload fraction m_payload/m0
`R`	mass ratio of each stage
`gross_per_payload`	lift-off mass per unit payload mass

- **Method:** $R = exp$(dv/(n Isp g0)); lambda = (1 - eps R)/(R - 1); payload fraction = (lambda/(1 + lambda))^n
- **Assumes:** $1 \leq n \leq 6$; 0.01 <= eps <= 0.5; eps $R < 1$; identical stages, equal delta-v split
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_identical_stages_payload', n=1, dv=4078.4710789429123,
Isp=300.0, eps=0.1)
# -> lam = 0.2, payload_fraction = 0.166667, R = 4, gross_per_payload = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Identical stages: n = 1, dv = 4078.4710789429123 m/s, Isp = 300 s, eps = 0.1	SOLVED	lam = 0.2, payload_fraction = 0.166667, R = 4, gross_per_payload = 6
Identical stages: n = 1, dv = 20 km/s, Isp = 300 s, eps = 0.1	INVALID_INPUT	Payload fraction of a rocket with identical stages: the velocity change is not reachable with this structural coefficien
Identical stages: n = 2, dv = 9 km/s, Isp = 350 s	NEEDS_INPUT	aero_identical_stages_payload needs eps (structural coefficient ms/(ms + mp) of every stage). Example: solve('aero_ident

Circular orbit at an altitude above the Earth — `aero_circular_orbit`

Orbital speed, period and revolutions per day of a circular orbit at an altitude above the Earth's equatorial radius.

$$v = \sqrt{\frac{\mu}{R+h}}, \quad T = 2\pi\sqrt{\frac{(R+h)^3}{\mu}}$$

input	meaning	condition
`h`	orbit altitude above the Earth (km)	

output	meaning
v	circular orbital speed

output	meaning
`T`	orbital period
`T_min`	period in minutes
`revs_per_day`	revolutions per day (86400 s)

- **Method:** $r = R + h$; $v = \sqrt{\frac{\mu}{r}}$; $T = 2$ pi sqrt(r^3/mu)
- **Assumes:** 100 $km \le h \le 1e6$ km; two-body Earth, $mu = 3.986004418e14\ m^3/s^2$, $R = 6378.1366$ km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_circular_orbit', h=19134409.799999997)
# -> v = 3952.68 m/s, T = 40554.7 s, T_min = 675.912, revs_per_day = 2.13045   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Circular orbit: h = 19134.409799999998 km	SOLVED	v = 3952.68 m/s, T = 40554.7 s, T_min = 675.912, revs_per_day = 2.13045
Circular orbit: h = 50 km	INVALID_INPUT	Circular orbit at an altitude above the Earth: the altitude must lie between 100 km and 1e+06 km.
Circular orbit:	NEEDS_INPUT	aero_circular_orbit needs h (orbit altitude above the Earth (km)). Example: solve('aero_circular_orbit', h='1.91344e+07

Escape speed at an altitude above the Earth — `aero_escape_speed`

The speed needed to escape the Earth's gravity from an altitude, the circular orbital speed there and the burn needed to escape from that circular orbit.

$$v_{esc} = \sqrt{\frac{2\mu}{R+h}} = \sqrt{2}\, v_{circ}$$

input	meaning	condition
`h`	altitude above the Earth (km); 0 = the surface	

output	meaning
`v_esc`	escape speed
`v_circ`	circular orbital speed at that altitude
`dv`	burn from the circular orbit to escape

- **Method:** v_esc = sqrt(2 mu/(R + h)); $v_{circ} = \sqrt{\frac{\mu}{R+h}}$
- **Assumes:** $0 \le h \le 1e6$ km; two-body Earth, no atmosphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_escape_speed', h=0.0)
# -> v_esc = 11179.9 m/s, v_circ = 7905.37 m/s, dv = 3274.51 m/s   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_escape_speed', h=6378136.6)
# -> v_esc = 7905.37 m/s, v_circ = 5589.94 m/s, dv = 2315.43 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Escape speed: h = 0 km	SOLVED	v_esc = 11179.9 m/s, v_circ = 7905.37 m/s, dv = 3274.51 m/s
Escape speed: h = -10 km	INVALID_INPUT	Escape speed at an altitude above the Earth: the altitude must lie between 0 and 1e6 km.
Escape speed:	NEEDS_INPUT	aero_escape_speed needs h (altitude above the Earth (km); 0 = the surface). Example: solve('aero_escape_speed', h='0 m')

Circular-orbit altitude for an orbital period — `aero_altitude_for_period`

The altitude above the Earth of the circular orbit with a given period (for example a geostationary orbit, one sidereal day), and its orbital speed.

$$R + h = \left(\mu \frac{T^2}{4\pi^2} \right)^{1/3}$$

input	meaning	condition
`T`	orbital period (s, min, h, day)	

output	meaning
`h`	orbit altitude
`h_km`	altitude in km
`v`	orbital speed

- **Method:** r = (mu (T/(2 pi))^2)^(1/3); $h = r - R$
- **Assumes:** the altitude between 100 km and 1e6 km; two-body Earth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_altitude_for_period', T=40554.74657601441)
# -> h = 1.91344e+07 m, h_km = 19134.4, v = 3952.68 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Altitude for period: T = 40554.74657601441 s	SOLVED	h = 1.91344e+07 m, h_km = 19134.4, v = 3952.68 m/s
Altitude for period: T = 60 min	INVALID_INPUT	Circular-orbit altitude for an orbital period: the altitude must lie between 100 km and 1e+06 km.

question	outcome	result
Altitude for period:	NEEDS_INPUT	aero_altitude_for_period needs T (orbital period (s, min, h, day)). Example: solve('aero_altitude_for_period', T='40554.

Bi-elliptic transfer between circular orbits — `aero_bielliptic_transfer`

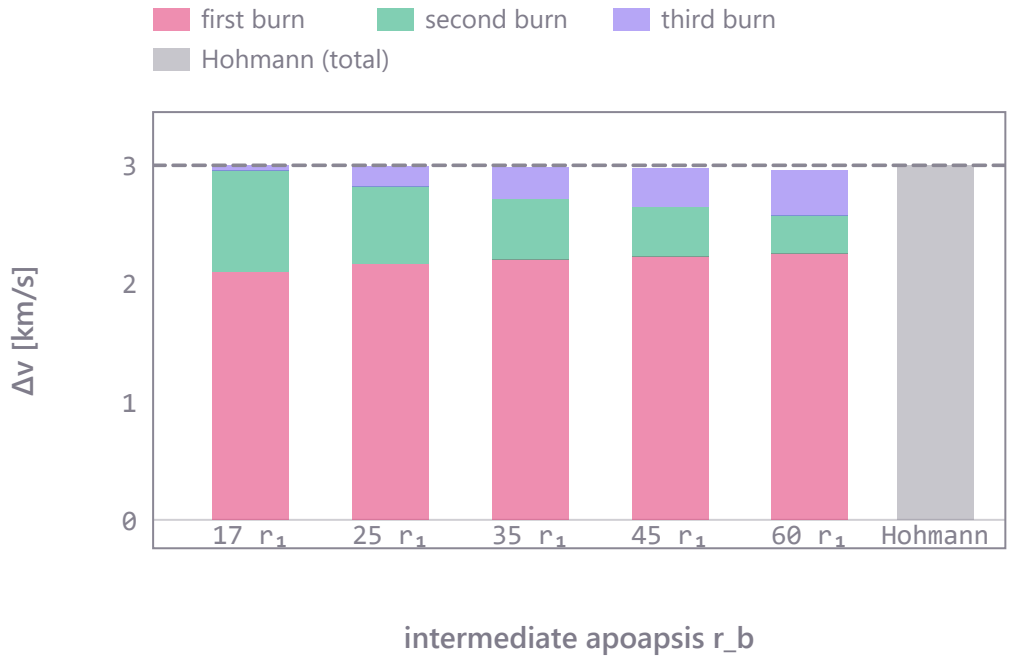
The three burns, total delta-v and transfer time of a bi-elliptic transfer from a lower to a higher circular orbit around the Earth through an intermediate apoapsis, with the Hohmann delta-v for comparison.

$$v = \sqrt{\mu\left(\frac{2}{r} - \frac{1}{a}\right)}, \quad t = \pi\left(\sqrt{\frac{a_1^3}{\mu}} + \sqrt{\frac{a_2^3}{\mu}}\right)$$

input	meaning	condition
`h1`	altitude of the initial circular orbit	
`h2`	altitude of the final circular orbit	
`hb`	altitude of the intermediate apoapsis (>= h2)	

output	meaning
`dv1`	first burn
`dv2`	second burn (at the intermediate apoapsis)
`dv3`	third burn (circularization)
`dv`	total delta-v
`t`	total transfer time
`t_days`	transfer time in days
`dv_hohmann`	total delta-v of the Hohmann transfer between the same orbits

- **Method:** two transfer ellipses (r1, rb) and (r2, rb): burns from vis-viva at r1, rb and r2; $t = pi$ (sqrt(a1^3/mu) + sqrt(a2^3/mu))
- **Assumes:** 100 $km \le h_1 < h_2 \le hb$; coplanar circular orbits, impulsive burns, two-body Earth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The three burns of bi-elliptic transfers from a circular orbit of r_1 = 12756.3 km to 15 r_1 around the Earth, stacked, for five intermediate apoapses, against the Hohmann transfer's 2.99743 km/s, all from the package; totals r_b = 17 r_1: 2.99791 km/s; r_b = 25 r_1: 2.9904 km/s; r_b = 35 r_1: 2.97848 km/s; r_b = 45 r_1: 2.96888 km/s; r_b = 60 r_1: 2.95846 km/s.

```
aerospace_engineering_ai999x.solve('aero_bielliptic_transfer', h1=6378136.6, h2=19134409.799999997,
hb=19134409.799999997)
# -> dv1 = 864.766 m/s, dv2 = 725.331 m/s, dv3 = 0 m/s, dv = 1590.1 m/s, t = 33447.9 s, t_days =
0.387129, dv_hohmann = 1590.1 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bi-elliptic transfer: h1 = 300 km, h2 = 200 km, hb = 400 km	INVALID_INPUT	Bi-elliptic transfer between circular orbits: the bi-elliptic transfer here is outward: h1 < h2 <= hb.
Bi-elliptic transfer: h1 = 300 km, h2 = 100000 km	NEEDS_INPUT	aero_bielliptic_transfer needs hb (altitude of the intermediate apoapsis (>= h2)). Example: solve('aero_bielliptic_trans

Lambert's problem (orbit through two positions in a given time) — `aero_lambert_problem`

The velocities at departure and arrival of the zero-revolution Earth orbit that goes from one position to another in a given time of flight (prograde or retrograde), with the transfer angle.

$$\sqrt{\mu}\,t = \left(\frac{y}{C(z)}\right)^{3/2} S(z) + A\sqrt{y}, \quad \mathbf{v}_1 = \frac{\mathbf{r}_2 - f\mathbf{r}_1}{g}, \quad \mathbf{v}_2 = \frac{\dot{g}\mathbf{r}_2 - \mathbf{r}_1}{g}$$

input	meaning	condition
`r1`	departure position vector in km, e.g. '[7000, 0, 0]'	
`r2`	arrival position vector in km	
`tof`	time of flight	
`direction`	prograde (default) or retrograde transfer	

output	meaning
`v1`	departure velocity vector in km/s
`v2`	arrival velocity vector in km/s
`dtheta_deg`	transfer angle in degrees

- **Method:** universal variables: solve sqrt(mu) $t = (y/C)^{1.5}$ S + A sqrt(y) for z by bisection (Stumpff functions C, S); $f = 1 - y/r_1$, $g = A$ sqrt(y/mu), g' = $1 - y/r_2$; v1 = (r2 - f r1)/g, v2 = (g' r2 - r1)/g
- **Assumes:** 2 deg <= transfer angle <= 358 deg, not within 2 deg of 180; tof > 0; zero revolutions, two-body Earth (mu = 398600.4418 km^3/s^2)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_lambert_problem', r1='7000, 0, 0', r2='0, 7000, 0',
tof=1457.1291594215038)
# -> v1 = [0.0, 7.54605329011, 0.0], v2 = [-7.54605329011, 0.0, 0.0], dtheta_deg = 90   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lambert problem: r1 = [7000, 0, 0], r2 = [0, 7000, 0], tof = 1457.1291594215038 s	SOLVED	v1 = [0.0, 7.54605329011, 0.0], v2 = [-7.54605329011, 0.0, 0.0], dtheta_deg = 90
Lambert problem: r1 = [7000, 0, 0], r2 = [-7000, 1, 0], tof = 1 h	INVALID_INPUT	Lambert's problem (orbit through two positions in a given time): the transfer angle must be between 2 and 358 degrees an
Lambert problem: r1 = [7000, 0, 0], r2 = [0, 7000, 0]	NEEDS_INPUT	aero_lambert_problem needs tof (time of flight). Example: solve('aero_lambert_problem', r1='7000, 0, 0', r2='0, 7000, 0'

Nodal regression from the Earth's oblateness (J2) — `aero_j2_nodal_regression`

The secular drift of the right ascension of the ascending node caused by the Earth's equatorial bulge (J2), from the perigee and apogee altitudes and the inclination.

$$\dot{\Omega} = -\frac{3}{2}\, n\, J_2 \left(\frac{R}{p}\right)^2 \cos i$$

input	meaning	condition
`hp`	perigee altitude (or the altitude of a circular orbit)	
`i`	inclination (deg)	
`ha`	apogee altitude (default: circular orbit)	

output	meaning
`raan_rate_deg_day`	node drift rate in degrees per day (negative = westward regression)
`raan_rate`	node drift rate

- **Method:** dOmega/dt = -$(3/2)$ n J2 $(R/p)^2$ cos i, $n = \sqrt{\frac{\mu}{a^3}}$, $p = a$(1 - e^2)
- **Assumes:** 100 $km \le hp \le ha \le 1e5$ km; 1 deg <= i <= 179 deg; first-order secular J2 theory, $J_2 = 1.08263e-3$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_j2_nodal_regression', hp=19134409.799999997,
i=1.0471975511965976)
# -> raan_rate_deg_day = -0.0389221, raan_rate = -7.86248e-09 rad/s   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_j2_nodal_regression', hp=19134409.799999997,
i=1.5707963267948966)
# -> raan_rate_deg_day = 0, raan_rate = 0 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nodal regression: hp = 19134.409799999998 km, i = 60 deg	SOLVED	raan_rate_deg_day = -0.0389221, raan_rate = -7.86248e-09 rad/s
Nodal regression: hp = 700 km, i = 200 deg	INVALID_INPUT	Nodal regression from the Earth's oblateness (J2): the inclination must lie between 1 and 179 degrees (the node and peri
Nodal regression: hp = 700 km	NEEDS_INPUT	aero_j2_nodal_regression needs i (inclination (deg)). Example: solve('aero_j2_nodal_regression', hp='1.91344e+07 m', i='

Rotation of the line of apsides from J2 — `aero_j2_apsidal_rotation`

The secular rotation of the argument of perigee of an eccentric Earth orbit caused by J2 (zero at the critical inclination 63.4 degrees, as used by Molniya orbits).

$$\dot{\omega} = \frac{3}{4}\, n\, J_2 \left(\frac{R}{p}\right)^2 \left(5\cos^2 i - 1\right)$$

input	meaning	condition
`hp`	perigee altitude	
`ha`	apogee altitude	
`i`	inclination (deg)	

output	meaning
`argp_rate_deg_day`	perigee rotation rate in degrees per day
`argp_rate`	perigee rotation rate

- **Method:** domega/dt = (3/4) n J2 $(R/p)^2$ (5 cos^2 i - 1)
- **Assumes:** 100 $km \le hp < ha \le 1e5$ km; eccentricity >= 1e-4; 1 deg <= i <= 179 deg; first-order secular J2 theory
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_j2_apsidal_rotation', hp=12756273.2, ha=25512546.4,
i=1.5707963267948966)
# -> argp_rate_deg_day = -0.0442847, argp_rate = -8.94575e-09 rad/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apsidal rotation: hp = 500 km, ha = 500 km, i = 63.4 deg	INVALID_INPUT	Rotation of the line of apsides from J2: the orbit must be eccentric (apogee above perigee) for its perigee to be define
Apsidal rotation: hp = 500 km, ha = 39800 km	NEEDS_INPUT	aero_j2_apsidal_rotation needs i (inclination (deg)). Example: solve('aero_j2_apsidal_rotation', hp='1.27563e+07 m', ha=

Inclination of a sun-synchronous orbit — `aero_sun_synchronous_inclination`

The inclination at which J2 makes a circular orbit's plane turn eastward at one revolution per year (sun-synchronous), at an altitude, with the orbital period.

$$\cos i = -\frac{\dot{\Omega}_{\odot}}{\frac{3}{2}\, n\, J_2\, (R/r)^2}$$

input	meaning	condition
`h`	altitude of the circular orbit (km)	

output	meaning
`i_deg`	sun-synchronous inclination in degrees (retrograde, > 90)
`period_min`	orbital period in minutes

- **Method:** cos i = -(dOmega/dt)_sun / ((3/2) n J2 (R/r)^2)
- **Assumes:** 100 $km \le h \le 6000$ km (none exists above about 5970 km); circular orbit, first-order J2, node rate 360 deg per 365.2421897 days
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_sun_synchronous_inclination', h=700000.0)
# -> i_deg = 98.188, period_min = 98.773   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sun-synchronous orbit: h = 700 km	SOLVED	i_deg = 98.188, period_min = 98.773
Sun-synchronous orbit: h = 7000 km	INVALID_INPUT	Inclination of a sun-synchronous orbit: the altitude must lie between 100 km and 6000 km.
Sun-synchronous orbit:	NEEDS_INPUT	aero_sun_synchronous_inclination needs h (altitude of the circular orbit (km)). Example: solve('aero_sun_synchronous_inc

Peak deceleration of a ballistic re-entry (Allen-Eggers) — `aero_ballistic_entry_peak_deceleration`

The peak deceleration (in g), the altitude where it occurs and the speed there for a body entering an exponential atmosphere on a straight ballistic path, from its mass, drag coefficient and reference area (ballistic coefficient m/(CD A)), entry speed and flight-path angle.

$$a_{max} = \frac{V_E^2 \sin\gamma_E}{2eH}, \quad h^* = H \ln\frac{\rho_0 H}{\beta \sin\gamma_E}, \quad V^* = V_E\, e^{-1/2}, \quad \beta = \frac{m}{C_D A}$$

input	meaning	condition
`m`	mass	
`CD`	drag coefficient	
`A`	drag reference area	
`VE`	entry speed	
`gamma_e`	entry flight-path angle below the horizontal (deg)	
`H`	atmospheric scale height (default 7.2 km)	
`rho0`	surface density of the exponential atmosphere (default 1.225 kg/m^3)	

output	meaning
`n_max`	peak deceleration in g (9.80665 m/s^2)
`a_max`	peak deceleration
`h_peak`	altitude of peak deceleration
`V_peak`	speed at peak deceleration

- **Method:** beta = m/(CD A); a_max = VE^2 sin(γ)/(2 e H); h* = H ln(rho0 H/(beta sin gamma)); V* = VE $e^{-1/2}$
- **Assumes:** 2 deg <= gamma_e <= 90 deg; peak above sea level; straight-line path, gravity and lift neglected, exponential atmosphere rho0 exp(-h/H)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_ballistic_entry_peak_deceleration', m=1000.0, CD=1.0,
A=1.0, VE=7000.0, gamma_e=1.5707963267948966)
# -> n_max = 127.649, a_max = 1251.81 m/s^2, h_peak = 15674.6 m, V_peak = 4245.71 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Peak deceleration: m = 1000 kg, CD = 1, A = 1 m^2, VE = 7 km/s, gamma_e = 90 deg	SOLVED	n_max = 127.649, a_max = 1251.81 m/s^2, h_peak = 15674.6 m, V_peak = 4245.71 m/s
Peak deceleration: m = 1000000 kg, CD = 1, A = 1 m^2, VE = 7 km/s, gamma_e = 90 deg	INVALID_INPUT	Peak deceleration of a ballistic re-entry (Allen-Eggers): the deceleration would peak below sea level (the body reaches
Peak deceleration: m = 1000 kg, CD = 1, A = 1 m^2, VE = 7 km/s	NEEDS_INPUT	aero_ballistic_entry_peak_deceleration needs gamma_e (entry flight-path angle below the horizontal (deg)). Example: solv

Speed and deceleration of a ballistic re-entry at an altitude — `aero_ballistic_entry_velocity`

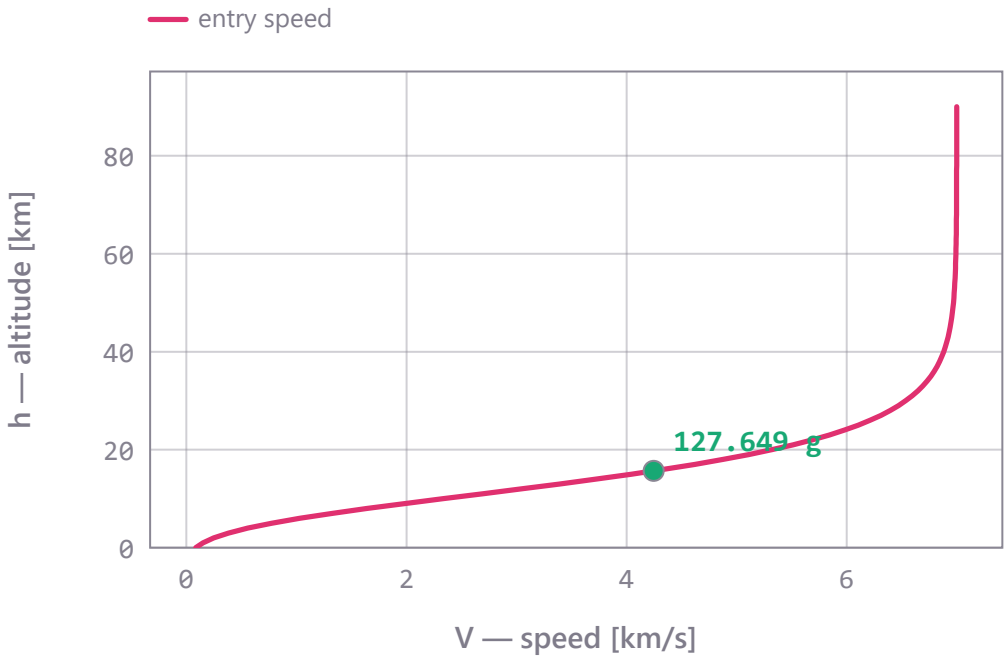
The speed and the deceleration of a body on a straight ballistic entry into an exponential atmosphere when it reaches a given altitude (Allen-Eggers).

$$V = V_E \exp\left(-\frac{\rho_0 H e^{-h/H}}{2\beta \sin\gamma_E}\right), \quad a = \frac{\rho V^2}{2\beta}$$

input	meaning	condition
`m`	mass	
`CD`	drag coefficient	
`A`	drag reference area	
`VE`	entry speed	
`gamma_e`	entry flight-path angle below the horizontal (deg)	
`h`	altitude	
`H`	atmospheric scale height (default 7.2 km)	
`rho0`	surface density (default 1.225 kg/m^3)	

output	meaning
`V`	speed at the altitude
`n`	deceleration in g
`a`	deceleration

- **Method:** $V = VE$ exp(-rho0 H e^(-h/H)/(2 beta sin gamma)); $a = \rho$ V^2/(2 beta), beta = m/(CD A)
- **Assumes:** 2 deg <= gamma_e <= 90 deg; $0 \le h \le 40$ H; straight-line path, gravity and lift neglected, exponential atmosphere
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Speed against altitude of a ballistic entry (m = 1000 kg, C_D = 1, A = 1 m², entry speed 7 km/s, vertical), 91 altitudes computed by the package; marked: peak deceleration 127.649 g at h = 15.6746 km, V = 4.24571 km/s.

```
aerospace_engineering_ai999x.solve('aero_ballistic_entry_velocity', m=1000.0, CD=1.0, A=1.0,
VE=7000.0, gamma_e=1.5707963267948966, h=15674.557464134641)
# -> V = 4245.71 m/s, n = 127.649, a = 1251.81 m/s^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ballistic re-entry at altitude: m = 1000 kg, CD = 1, A = 1 m^2, VE = 7 km/s, gamma_e = 90 deg, h = 500 km	INVALID_INPUT	Speed and deceleration of a ballistic re-entry at an altitude: the altitude must lie between sea level and 40 scale heig
Ballistic re-entry at altitude: m = 1000 kg, CD = 1, A = 1 m^2, VE = 7 km/s, gamma_e = 90 deg	NEEDS_INPUT	aero_ballistic_entry_velocity needs h (altitude). Example: solve('aero_ballistic_entry_velocity', m='1000 kg', CD=1, A='

Best lift coefficients and lift-to-drag ratios of a drag polar — `aero_drag_polar_optimum`

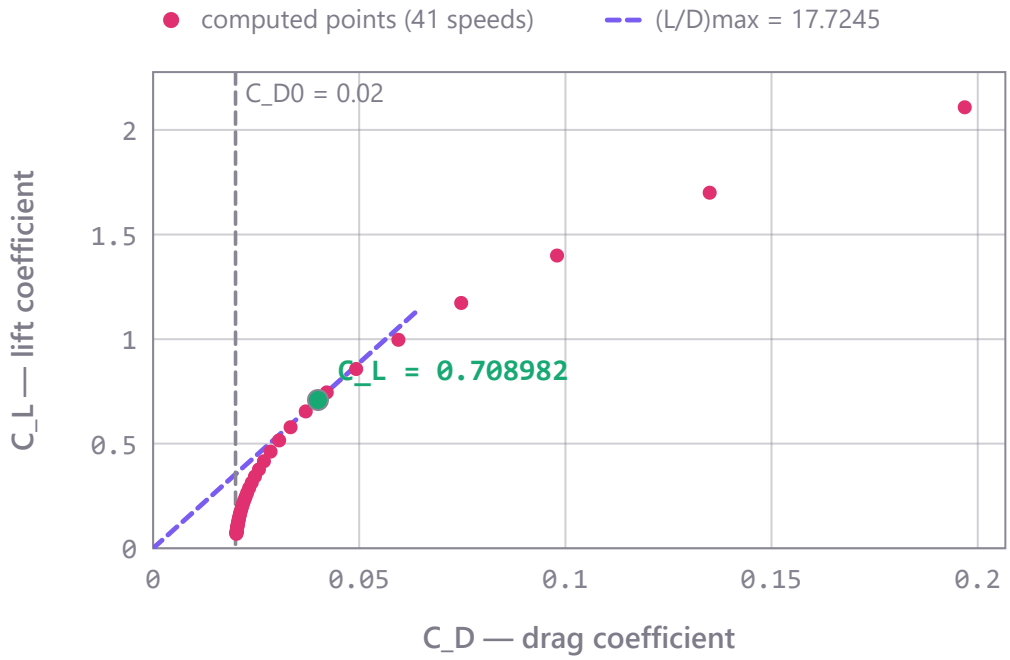
For a parabolic drag polar CD = CD0 + CL^2/(pi e AR): the lift coefficient and L/D for minimum drag (maximum L/D), for minimum power (best propeller endurance) and for the jet's best range (maximum sqrt(CL)/CD).

$$C_{L,md} = \sqrt{\frac{C_{D0}}{K}}, \quad \left(\frac{L}{D}\right)_{max} = \frac{1}{2\sqrt{C_{D0}K}}, \quad K = \frac{1}{\pi e AR}$$

input	meaning	condition
`CD0`	zero-lift (parasite) drag coefficient	
`AR`	wing aspect ratio b^2/S	
`e`	Oswald span efficiency factor	

output	meaning
`CL_md`	lift coefficient for minimum drag
`LD_max`	maximum lift-to-drag ratio
`CL_mp`	lift coefficient for minimum power required
`LD_mp`	L/D at minimum power
`CL_mr`	lift coefficient for best jet range (max sqrt(CL)/CD)
`LD_mr`	L/D at best jet range

- **Method:** K = 1/(pi e AR); $CL_{md} = \sqrt{\frac{CD_0}{K}}$, (L/D)max = 1/(2 sqrt(CD0 K)); $CL_mp = sqrt(3\ CD0/K)$, $CD = 4$ CD0; $CL_mr = sqrt(CD0/(3K))$, $CD = 4/3$ CD0
- **Assumes:** $0.002 \le CD0 \le 0.2$; $1 \le AR \le 50$; $0.3 \le e \le 1$; parabolic drag polar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Drag polar C_D = C_D0 + C_Di for C_D0 = 0.02, aspect ratio 10 and Oswald factor 0.8: the induced drag at 41 speeds from the package (W = 10000 N, S = 16 m², sea level); the tangent from the origin touches at the best lift-to-drag point it computes — marked: C_D = 0.04, C_L = 0.708982, L/D = 17.7245.

```
aerospace_engineering_ai999x.solve('aero_drag_polar_optimum', CD0=0.02, AR=10.0, e=0.8)
# -> CL_md = 0.708982, LD_max = 17.7245, CL_mp = 1.22799, LD_mp = 15.3499, CL_mr = 0.409331, LD_mr
= 15.3499   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Drag polar: CD0 = 0.02, AR = 10, e = 0.8	SOLVED	CL_md = 0.708982, LD_max = 17.7245, CL_mp = 1.22799, LD_mp = 15.3499, CL_mr = 0.409331, LD_mr = 15.3499
Drag polar: CD0 = 0.02, AR = 10, e = 1.5	INVALID_INPUT	Best lift coefficients and lift-to-drag ratios of a drag polar: the Oswald efficiency factor must lie between 0.3 and 1.
Drag polar: CD0 = 0.02, AR = 10	NEEDS_INPUT	aero_drag_polar_optimum needs e (Oswald span efficiency factor). Example: solve('aero_drag_polar_optimum', CD0=0.02, AR=

Speeds for minimum drag, minimum power and best jet range — `aero_min_drag_power_speeds`

In steady level flight at an altitude: the speed for minimum drag and the minimum drag, the speed for minimum power required and that power, and the jet's best-range speed, for a parabolic drag polar.

$$V_{md} = \sqrt{\frac{2W}{\rho S}\sqrt{\frac{K}{C_{D0}}}}, \quad D_{min} = 2W\sqrt{KC_{D0}}, \quad V_{mp} = \frac{V_{md}}{3^{1/4}}$$

input	meaning	condition
`W`	aircraft weight (N, kN, lbf)	
`S`	wing area (m^2, ft^2)	
`h`	altitude (standard atmosphere)	

input	meaning	condition
`CD0`	zero-lift drag coefficient	
`AR`	aspect ratio	
`e`	Oswald efficiency factor	

output	meaning
`V_md`	speed for minimum drag (max L/D)
`D_min`	minimum drag
`V_mp`	speed for minimum power required
`P_min`	minimum power required
`V_mr`	best-range speed of a jet (max V/D)

- **Method:** $V_md = sqrt$(2W/(rho S) sqrt(K/CD0)); D_min = 2W sqrt(K CD0); $V_mp = V_md/3^{1/4}$; $V_mr = 3^{1/4}$ V_md
- **Assumes:** W, $S > 0$; -1 $km \le h \le 47$ km; parabolic drag polar; steady level flight, $L = W$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_min_drag_power_speeds', W=10000.0, S=20.0, h=0.0,
CD0=0.02, AR=10.0, e=0.8)
# -> V_md = 33.9324 m/s, D_min = 564.19 N, V_mp = 25.7831 m/s, P_min = 16796.9 W, V_mr = 44.6576
m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Speeds for minimum drag: W = 10 kN, S = 20 m^2, h = 0 m, CD0 = 0.02, AR = 10, e = 0.8	SOLVED	V_md = 33.9324 m/s, D_min = 564.19 N, V_mp = 25.7831 m/s, P_min = 16796.9 W, V_mr = 44.6576 m/s
Speeds for minimum drag: W = -10 kN, S = 20 m^2, h = 0 m, CD0 = 0.02, AR = 10, e = 0.8	INVALID_INPUT	Speeds for minimum drag, minimum power and best jet range: the weight and the wing area must be a positive number.
Speeds for minimum drag: W = 10 kN, S = 20 m^2, CD0 = 0.02, AR = 10, e = 0.8	NEEDS_INPUT	aero_min_drag_power_speeds needs h (altitude (standard atmosphere)). Example: solve('aero_min_drag_power_speeds', W='100

Breguet range of a jet aircraft — `aero_breguet_range_jet`

The cruise range and flight time of a jet aircraft at constant speed and lift-to-drag ratio (cruise-climb) from its thrust-specific fuel consumption and the weights at the start and end of cruise.

$$R = \frac{V}{g_0 c}\,\frac{L}{D}\ln\frac{W_0}{W_1}$$

input	meaning	condition
`V`	cruise true airspeed	
`c`	thrust-specific fuel consumption (kg/(N*s), $1/h$, lb/(lbf*h))	

input	meaning	condition
`LD`	lift-to-drag ratio	
`W0`	weight at the start of cruise	
`W1`	weight at the end of cruise	

output	meaning
`R`	range
`R_km`	range in km
`t`	flight time
`t_h`	flight time in hours

- **Method:** R = (V/(g0 c)) (L/D) ln(W0/W1), c the mass-flow TSFC
- **Assumes:** $V > 0$; $0 < W_1 < W_0$; $1 \le L/D \le 60$; constant V, L/D and TSFC (cruise-climb)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_breguet_range_jet', V=200.0, c=2.832545036049801e-05,
LD=15.0, W0=271828.1828459045, W1=100000.0)
# -> R = 1.08e+07 m, R_km = 10800, t = 54000 s, t_h = 15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Breguet range: V = 200 m/s, c = 1 1/h, LD = 15, W0 = 271.8281828459045 kN, W1 = 100 kN	SOLVED	R = 1.08e+07 m, R_km = 10800, t = 54000 s, t_h = 15
Breguet range: V = 200 m/s, c = 0.6 1/h, LD = 15, W0 = 100 kN, W1 = 150 kN	INVALID_INPUT	Breguet range of a jet aircraft: the final weight must be below the initial weight.
Breguet range: V = 200 m/s, c = 0.6 1/h, LD = 15, W0 = 100 kN	NEEDS_INPUT	aero_breguet_range_jet needs W1 (weight at the end of cruise). Example: solve('aero_breguet_range_jet', V='200 m/s', c='

Endurance of a jet aircraft — `aero_breguet_endurance_jet`

The time a jet aircraft can stay airborne at constant lift-to-drag ratio from its thrust-specific fuel consumption and initial and final weights (independent of speed).

$$E = \frac{1}{g_0 c}\,\frac{L}{D}\ln\frac{W_0}{W_1}$$

input	meaning	condition
`c`	thrust-specific fuel consumption (kg/(N*s), $1/h$, lb/(lbf*h))	
`LD`	lift-to-drag ratio	
`W0`	initial weight	
`W1`	final weight	

output	meaning
`E`	endurance
`E_h`	endurance in hours

- **Method:** E = (1/(g0 c)) (L/D) ln(W0/W1)
- **Assumes:** $0 < W_1 < W_0$; $1 \leq L/D \leq 60$; constant L/D and TSFC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_breguet_endurance_jet', c=2.832545036049801e-05, LD=15.0,
W0=271828.1828459045, W1=100000.0)
# -> E = 54000 s, E_h = 15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Jet endurance: c = 1 1/h, LD = 15, W0 = 271.8281828459045 kN, W1 = 100 kN	SOLVED	E = 54000 s, E_h = 15
Jet endurance: c = 0.6 1/h, LD = 100, W0 = 300 kN, W1 = 240 kN	INVALID_INPUT	Endurance of a jet aircraft: the lift-to-drag ratio must lie between 1 and 60.
Jet endurance: c = 0.6 1/h, LD = 15, W0 = 300 kN	NEEDS_INPUT	aero_breguet_endurance_jet needs W1 (final weight). Example: solve('aero_breguet_endurance_jet', c='2.83255e-05 kg/(N*s)

Fuel a jet needs for a cruise range — `aero_jet_fuel_for_range`

The fuel weight (and mass) a jet aircraft burns cruising a distance at constant speed and L/D, and its weight at the end, from the Breguet relation.

$$W_1 = W_0 \exp\left(-\frac{R\, g_0 c}{V\,(L/D)}\right)$$

input	meaning	condition
`R`	cruise distance (km, nmi)	
`V`	cruise speed	
`c`	thrust-specific fuel consumption	
`LD`	lift-to-drag ratio	
`W0`	weight at the start of cruise	

output	meaning
`W1`	weight at the end of cruise
`Wf`	fuel weight burned
`mf`	fuel mass burned

- **Method:** $W_1 = W_0$ exp(-R g0 c/(V L/D)); $Wf = W_0 - W_1$
- **Assumes:** R, $V > 0$; fuel at most 80% of W0; constant V, L/D and TSFC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_jet_fuel_for_range', R=10800000.0, V=200.0,
c=2.832545036049801e-05, LD=15.0, W0=271828.1828459045)
# -> W1 = 100000 N, Wf = 171828 N, mf = 17521.6 kg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fuel for range: R = 10800 km, V = 200 m/s, c = 1 1/h, LD = 15, W0 = 271.8281828459045 kN	SOLVED	W1 = 100000 N, Wf = 171828 N, mf = 17521.6 kg
Fuel for range: R = 100000 km, V = 200 m/s, c = 1 1/h, LD = 15, W0 = 100 kN	INVALID_INPUT	Fuel a jet needs for a cruise range: the fuel needed would exceed 80% of the initial weight.
Fuel for range: R = 1000 km, V = 200 m/s, c = 1 1/h, LD = 15	NEEDS_INPUT	aero_jet_fuel_for_range needs W0 (weight at the start of cruise). Example: solve('aero_jet_fuel_for_range', R='1.08e+07

Jet range at constant altitude and lift coefficient — `aero_jet_range_constant_altitude`

The range and flight time of a jet cruising at a constant altitude and constant lift coefficient, so that its speed falls as fuel burns (the second Breguet form).

$$R = \frac{2}{g_0 c}\sqrt{\frac{2}{\rho S}}\,\frac{C_L^{1/2}}{C_D}\left(\sqrt{W_0}-\sqrt{W_1}\right)$$

input	meaning	condition
`W0`	initial weight	
`W1`	final weight	
`S`	wing area	
`h`	cruise altitude	
`CL`	cruise lift coefficient	
`CD`	cruise drag coefficient	
`c`	thrust-specific fuel consumption	

output	meaning
`R`	range
`R_km`	range in km
`t`	flight time
`t_h`	flight time in hours

- **Method:** R = (2/(g0 c)) sqrt(2/(rho S)) (sqrt(CL)/CD) (sqrt(W0) - sqrt(W1)); t = (CL/CD)/(g0 c) ln(W0/W1)
- **Assumes:** $0 < W_1 < W_0$; $0.05 \le CL \le 3$; $CD < CL$; constant altitude, CL and TSFC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_jet_range_constant_altitude', W0=400000.0, W1=100000.0,
S=100.0, h=0.0, CL=0.5, CD=0.05, c=1e-05)
# -> R = 1.16539e+07 m, R_km = 11653.9, t = 141363 s, t_h = 39.2674   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Jet range at constant altitude: W0 = 400 kN, W1 = 100 kN, S = 100 m^2, h = 0 m, CL = 0.5, CD = 0.05, c = 0.00001 kg/(N*s)	SOLVED	R = 1.16539e+07 m, R_km = 11653.9, t = 141363 s, t_h = 39.2674
Jet range at constant altitude: W0 = 400 kN, W1 = 100 kN, S = 100 m^2, h = 0 m, CL = 0.5, CD = 0.8, c = 0.00001 kg/(N*s)	INVALID_INPUT	Jet range at constant altitude and lift coefficient: the drag coefficient must lie between 0.003 and 0.5 and below CL.
Jet range at constant altitude: W0 = 400 kN, W1 = 100 kN, S = 100 m^2, CL = 0.5, CD = 0.05, c = 0.00001 kg/(N*s)	NEEDS_INPUT	aero_jet_range_constant_altitude needs h (cruise altitude). Example: solve('aero_jet_range_constant_altitude', W0='40000

Breguet range of a propeller aircraft — `aero_breguet_range_prop`

The cruise range of a propeller-driven aircraft at constant lift-to-drag ratio and propeller efficiency from its power-specific fuel consumption and initial and final weights.

$$R = \frac{\eta_p}{g_0 c_P} \frac{L}{D} \ln \frac{W_0}{W_1}$$

input	meaning	condition
`eta`	propeller efficiency	
`c`	power-specific fuel consumption (kg/J, $g/(kWh)$, $lb/(hph)$)	
`LD`	lift-to-drag ratio	
`W0`	initial weight	
`W1`	final weight	

output	meaning
`R`	range
`R_km`	range in km

- **Method:** R = (eta/(g0 c)) (L/D) ln(W0/W1), c the mass flow per shaft power
- **Assumes:** $0.1 \leq \eta \leq 1$; $0 < W_1 < W_0$; $1 \leq L/D \leq 60$; constant L/D, efficiency and BSFC
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_breguet_range_prop', eta=0.8, c=1e-07, LD=15.0,
W0=27182.818284590452, W1=10000.0)
# -> R = 1.22366e+07 m, R_km = 12236.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Propeller range: eta = 0.8, c = 0.0000001 kg/J, LD = 15, W0 = 27.18281828459045 kN, W1 = 10 kN	SOLVED	R = 1.22366e+07 m, R_km = 12236.6
Propeller range: eta = 1.5, c = 0.0000001 kg/J, LD = 15, W0 = 20 kN, W1 = 10 kN	INVALID_INPUT	Breguet range of a propeller aircraft: the propeller efficiency must lie between 0.1 and 1.
Propeller range: eta = 0.8, c = 0.0000001 kg/J, LD = 15, W0 = 20 kN	NEEDS_INPUT	aero_breguet_range_prop needs W1 (final weight). Example: solve('aero_breguet_range_prop', eta=0.8, c='1e-07 kg/J', LD=1

Endurance of a propeller aircraft at constant altitude — `aero_prop_endurance_constant_altitude`

How long a propeller-driven aircraft can loiter at a constant altitude and lift coefficient, from its propeller efficiency, power-specific fuel consumption, drag polar point, wing area and weights.

$$E = \frac{\eta_p}{g_0 c_P} \frac{C_L^{3/2}}{C_D} \sqrt{2\rho S} \left(W_1^{-1/2} - W_0^{-1/2} \right)$$

input	meaning	condition
`eta`	propeller efficiency	
`c`	power-specific fuel consumption	
`CL`	lift coefficient	
`CD`	drag coefficient	
`S`	wing area	
`h`	altitude	
`W0`	initial weight	
`W1`	final weight	

output	meaning
`E`	endurance
`E_h`	endurance in hours

- **Method:** E = (eta/(g0 c)) $(CL^{1.5}/CD)$ sqrt(2 rho S) (W1^(-1/2) - W0^(-1/2))
- **Assumes:** $0.1 \leq \eta \leq 1$; $0 < W_1 < W_0$; $0.05 \leq CL \leq 3$; constant altitude and CL
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_prop_endurance_constant_altitude', eta=0.8, c=1e-07,
CL=1.0, CD=0.05, S=20.0, h=0.0, W0=40000.0, W1=10000.0)
# -> E = 571041 s, E_h = 158.622   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Propeller endurance: eta = 0.8, c = 0.0000001 kg/J, CL = 1, CD = 0.05, S =	SOLVED	E = 571041 s, E_h = 158.622

question	outcome	result
20 m^2, h = 0 m, W0 = 40 kN, W1 = 10 kN		
Propeller endurance: eta = 0.8, c = 0.0000001 kg/J, CL = 1, CD = 0.05, S = 20 m^2, h = 0 m, W0 = 10 kN, W1 = 40 kN	INVALID_INPUT	Endurance of a propeller aircraft at constant altitude: the final weight must be below the initial weight.
Propeller endurance: eta = 0.8, c = 0.0000001 kg/J, CL = 1, CD = 0.05, S = 20 m^2, h = 0 m, W0 = 40 kN	NEEDS_INPUT	aero_prop_endurance_constant_altitude needs W1 (final weight). Example: solve('aero_prop_endurance_constant_altitude', e

Takeoff ground roll — `aero_takeoff_ground_roll`

The ground-roll distance and time to lift-off speed of an aircraft accelerating under constant thrust against aerodynamic drag and rolling friction on a level runway.

$$s = \frac{1}{2B}\ln\frac{A}{A - BV_{LO}^2}, \quad A = g\left(\frac{T}{W} - \mu\right), \quad B = \frac{g\rho S}{2W}(C_D - \mu C_L)$$

input	meaning	condition
`W`	weight	
`S`	wing area	
`h`	airfield altitude (standard atmosphere)	
`T`	thrust (constant during the roll)	
`mu`	rolling friction coefficient	
`CL`	lift coefficient during the ground run	
`CD`	drag coefficient during the ground run	
`VLO`	lift-off speed	

output	meaning
`s`	ground-roll distance
`t`	ground-roll time

- **Method:** $dV/dt = A - B\,V^2$, $A = g$ (T/W - mu), $B = g$ rho S (CD - mu CL)/(2W); $s = ln$(A/(A - B VLO^2))/(2B); t = artanh(VLO sqrt(B/A))/sqrt(A B)
- **Assumes:** thrust large enough to keep accelerating up to VLO; $0 \le mu \le 0.8$; constant thrust, level runway, no wind
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_takeoff_ground_roll', W=10000.0, S=10.0, h=0.0, T=3000.0,
mu=0.02, CL=1.0, CD=0.02, VLO=40.0)
# -> s = 291.347 m, t = 14.5674 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Takeoff roll: W = 10 kN, S = 10 m^2, h = 0 m, T = 3 kN, mu = 0.02, CL = 1, CD = 0.02, VLO = 40 m/s	SOLVED	s = 291.347 m, t = 14.5674 s
Takeoff roll: W = 10 kN, S = 10 m^2, h = 0 m, T = 0.1 kN, mu = 0.02, CL = 1, CD = 0.02, VLO = 40 m/s	INVALID_INPUT	Takeoff ground roll: the aircraft would stop accelerating before the lift-off speed (thrust too low).
Takeoff roll: W = 10 kN, S = 10 m^2, h = 0 m, T = 3 kN, mu = 0.02, CL = 1, CD = 0.02	NEEDS_INPUT	aero_takeoff_ground_roll needs VLO (lift-off speed). Example: solve('aero_takeoff_ground_roll', W='10000 N', S='10 m^2',

Landing ground roll — `aero_landing_ground_roll`

The ground-roll distance and time from touchdown to a stop of an aircraft braking on a level runway, with aerodynamic drag, lift (which unloads the wheels) and optional reverse thrust.

$$s = \frac{1}{2B}\ln\left(1 + \frac{BV_{TD}^2}{A}\right), \quad A = g\left(\frac{T_r}{W} + \mu\right), \quad B = \frac{g\rho S}{2W}(C_D - \mu C_L)$$

input	meaning	condition
`W`	weight at landing	
`S`	wing area	
`h`	airfield altitude	
`VTD`	touchdown speed	
`mu`	braking friction coefficient	
`CL`	lift coefficient during the roll	
`CD`	drag coefficient during the roll	
`Tr`	reverse thrust (default 0)	

output	meaning
`s`	ground-roll distance
`t`	ground-roll time

- **Method:** $-dV/dt = A + B\,V^2$, $A = g$ (Tr/W + mu), $B = g$ rho S (CD - mu CL)/(2W); $s = ln(1 + B$ VTD^2/A)/(2B); t = atan(VTD sqrt(B/A))/sqrt(A B)
- **Assumes:** $0 \le mu \le 0.8$; the net deceleration positive throughout; level runway, no wind, constant reverse thrust
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_landing_ground_roll', W=10000.0, S=10.0, h=0.0, VTD=30.0,
mu=0.4, CL=0.25, CD=0.1)
# -> s = 114.718 m, t = 7.64787 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Landing roll: W = 10 kN, S = 10 m^2, h = 0 m, VTD = 30 m/s, mu = 0.4, CL = 0.25, CD = 0.1	SOLVED	s = 114.718 m, t = 7.64787 s
Landing roll: W = 10 kN, S = 10 m^2, h = 0 m, VTD = 30 m/s, mu = 0.4, CL = 0.25, CD = 0.1, Tr = -5 kN	INVALID_INPUT	Landing ground roll: the reverse thrust must not be negative.
Landing roll: W = 10 kN, S = 10 m^2, h = 0 m, VTD = 30 m/s, mu = 0.4, CL = 0.25	NEEDS_INPUT	aero_landing_ground_roll needs CD (drag coefficient during the roll). Example: solve('aero_landing_ground_roll', W='1000

Level coordinated turn at a bank angle — `aero_level_turn_bank`

The load factor, turn radius, turn rate and the time for a full circle of an aircraft in a level, coordinated turn at a true airspeed and bank angle.

$$n = \frac{1}{\cos\phi}, \quad R = \frac{V^2}{g\tan\phi}, \quad \omega = \frac{g\tan\phi}{V}$$

input	meaning	condition
`V`	true airspeed	
`phi`	bank angle (deg)	
`g`	gravity (default 9.80665 m/s^2)	

output	meaning
`n`	load factor L/W
`phi_deg`	bank angle in degrees
`R`	turn radius
`omega`	turn rate
`omega_deg`	turn rate in degrees per second
`t_360`	time for a 360-degree turn

- **Method:** $n = \frac{1}{\cos(\phi)}$; R = V^2/(g tan(phi)); $\omega = g$ tan(phi)/V
- **Assumes:** 1 deg <= phi <= 85 deg; $V > 0$; level, coordinated turn at constant speed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_level_turn_bank', V=100.0, phi=0.7853981633974483)
# -> n = 1.41421, phi_deg = 45, R = 1019.72 m, omega = 0.0980665 rad/s, omega_deg = 5.6188, t_360 =
64.0707 s   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_level_turn_bank', V=50.0, phi=1.0471975511965976)
# -> n = 2, phi_deg = 60, R = 147.183 m, omega = 0.339712 rad/s, omega_deg = 19.4641, t_360 =
18.4956 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Level turn at bank angle: V = 100 m/s, phi = 45 deg	SOLVED	n = 1.41421, phi_deg = 45, R = 1019.72 m, omega = 0.0980665 rad/s, omega_deg = 5.6188, t_360 = 64.0707 s
Level turn at bank angle: V = 100 m/s, phi = 89 deg	INVALID_INPUT	Level coordinated turn at a bank angle: the bank angle must lie between 1 and 85 degrees.
Level turn at bank angle: V = 100 m/s	NEEDS_INPUT	aero_level_turn_bank needs phi (bank angle (deg)). Example: solve('aero_level_turn_bank', V='100 m/s', phi='0.785398 rad

Level turn at a load factor — `aero_level_turn_load_factor`

The bank angle, turn radius, turn rate and time for a full circle of a level coordinated turn flown at a true airspeed and load factor (g).

$$R = \frac{V^2}{g\sqrt{n^2-1}}, \quad \omega = \frac{g\sqrt{n^2-1}}{V}$$

input	meaning	condition
`V`	true airspeed	
`n`	load factor (g)	
`g`	gravity (default 9.80665 m/s^2)	

output	meaning
`n`	load factor
`phi_deg`	bank angle in degrees
`R`	turn radius
`omega`	turn rate
`omega_deg`	turn rate in degrees per second
`t_360`	time for a 360-degree turn

- **Method:** cos(phi) = 1/n; R = V^2/(g sqrt(n^2 - 1)); $\omega = g$ sqrt(n^2 - 1)/V
- **Assumes:** $1.0002 \le n \le 11.5$; $V > 0$; level, coordinated turn at constant speed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_level_turn_load_factor', V=100.0, n=2.0)
# -> n = 2, phi_deg = 60, R = 588.733 m, omega = 0.169856 rad/s, omega_deg = 9.73204, t_360 =
36.9912 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Level turn at load factor: V = 100 m/s, n = 2	SOLVED	n = 2, phi_deg = 60, R = 588.733 m, omega = 0.169856 rad/s, omega_deg = 9.73204, t_360 = 36.9912 s
Level turn at load factor: V = 100 m/s, n = 0.5	INVALID_INPUT	Level turn at a load factor: the load factor must lie between 1.0002 and 11.5.

question	outcome	result
Level turn at load factor: V = 100 m/s	NEEDS_INPUT	aero_level_turn_load_factor needs n (load factor (g)). Example: solve('aero_level_turn_load_factor', V='100 m/s', n=2).

Time to climb with a linearly decreasing rate of climb — `aero_time_to_climb`

The time to climb between two altitudes when the maximum rate of climb falls linearly from its sea-level value to zero at the absolute ceiling (the standard textbook model).

$$t = \int_{h_1}^{h_2} \frac{dh}{RC_0(1 - h/H_c)} = \frac{H_c}{RC_0} \ln \frac{H_c - h_1}{H_c - h_2}$$

input	meaning	condition
`RC0`	maximum rate of climb at sea level (m/s, ft/min)	
`Hc`	absolute ceiling	
`h1`	starting altitude	
`h2`	final altitude	

output	meaning
`t`	time to climb
`t_min`	time to climb in minutes

- **Method:** t = integral of $dh/RC = (Hc/RC0)$ ln$((Hc - h_1)/(Hc - h_2))$
- **Assumes:** $0 \le h_1 < h_2 < Hc$; RC(h) = RC0 $(1 - h/Hc)$ (stated linear model)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_time_to_climb', RC0=10.0, Hc=10000.0, h1=0.0,
h2=6321.205588285577)
# -> t = 1000 s, t_min = 16.6667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Time to climb: RC0 = 10 m/s, Hc = 10 km, h1 = 0 m, h2 = 6321.205588285577 m	SOLVED	t = 1000 s, t_min = 16.6667
Time to climb: RC0 = 10 m/s, Hc = 10 km, h1 = 0 m, h2 = 12 km	INVALID_INPUT	Time to climb with a linearly decreasing rate of climb: the altitudes must satisfy 0 <= h1 < h2 < absolute ceiling.
Time to climb: RC0 = 10 m/s, Hc = 10 km, h1 = 0 m	NEEDS_INPUT	aero_time_to_climb needs h2 (final altitude). Example: solve('aero_time_to_climb', RC0='10 m/s', Hc='10000 m', h1='0 m',

Maximum rate of climb of a jet — `aero_max_rate_of_climb_jet`

The maximum rate of climb of a jet aircraft (constant thrust) at an altitude, the speed at which it occurs and the climb angle there, for a parabolic drag polar.

$$V^2 = \frac{T + \sqrt{T^2 + 12ab}}{6a}, \quad RC_{max} = \frac{V(T - aV^2 - b/V^2)}{W}$$

input	meaning	condition
`W`	weight	
`S`	wing area	
`h`	altitude	
`T`	available thrust	
`CD0`	zero-lift drag coefficient	
`AR`	aspect ratio	
`e`	Oswald efficiency factor	

output	meaning
`RC_max`	maximum rate of climb
`V`	speed for the maximum rate of climb
`gamma_deg`	climb angle at that speed, degrees

- **Method:** $RC = V$ (T - D)/W with $D = a\,V^2 + b/V^2$, $a = \rho$ S $CD0/2$, $b = 2$ K W^2/(rho S); dRC/dV = 0 gives V^2 = (T + sqrt($T^2 + 12$ a b))/(6a)
- **Assumes:** minimum drag < T < 0.8 W; small climb angle (lift = weight); parabolic drag polar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_max_rate_of_climb_jet', W=100000.0, S=30.0, h=0.0,
T=30000.0, CD0=0.02, AR=7.0, e=0.8)
# -> RC_max = 31.1332 m/s, V = 167.945 m/s, gamma_deg = 10.6831   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum rate of climb of a jet: W = 100 kN, S = 30 m^2, h = 0 m, T = 1 kN, CD0 = 0.02, AR = 7, e = 0.8	INVALID_INPUT	Maximum rate of climb of a jet: the thrust is below the minimum drag: the aircraft cannot hold level flight.
Maximum rate of climb of a jet: W = 100 kN, S = 30 m^2, h = 0 m, CD0 = 0.02, AR = 7, e = 0.8	NEEDS_INPUT	aero_max_rate_of_climb_jet needs T (available thrust). Example: solve('aero_max_rate_of_climb_jet', W='100000 N', S='30

Steepest climb angle of a jet — `aero_max_climb_angle_jet`

The maximum climb angle of a jet aircraft (constant thrust, small-angle approximation lift = weight), the speed for it and the rate of climb at that speed.

$$\sin\gamma_{max} = \frac{T}{W} - \frac{1}{(L/D)_{max}}$$

input	meaning	condition
`W`	weight	
`S`	wing area	
`h`	altitude	
`T`	available thrust	
`CD0`	zero-lift drag coefficient	
`AR`	aspect ratio	
`e`	Oswald efficiency factor	

output	meaning
`gamma_max_deg`	maximum climb angle in degrees
`V`	speed for the steepest climb
`RC`	rate of climb at that speed

- **Method:** sin(gamma_max) = T/W - 2 sqrt(K CD0) at the minimum-drag speed
- **Assumes:** minimum drag < T < 0.8 W; lift = weight (small angle); parabolic drag polar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_max_climb_angle_jet', W=100000.0, S=30.0, h=0.0,
T=30000.0, CD0=0.02, AR=7.0, e=0.8)
# -> gamma_max_deg = 13.4482, V = 95.7843 m/s, RC = 22.2762 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum climb angle: W = 100 kN, S = 30 m^2, h = 0 m, T = 30 kN, CD0 = 0.02, AR = 7, e = 0.8	SOLVED	gamma_max_deg = 13.4482, V = 95.7843 m/s, RC = 22.2762 m/s
Maximum climb angle: W = 100 kN, S = 30 m^2, h = 0 m, T = 1 kN, CD0 = 0.02, AR = 7, e = 0.8	INVALID_INPUT	Steepest climb angle of a jet: the thrust must exceed the minimum drag and stay below 0.8 W.
Maximum climb angle: W = 100 kN, S = 30 m^2, h = 0 m, CD0 = 0.02, AR = 7, e = 0.8	NEEDS_INPUT	aero_max_climb_angle_jet needs T (available thrust). Example: solve('aero_max_climb_angle_jet', W='100000 N', S='30 m^2'

Maximum rate of climb of a propeller aircraft — `aero_max_rate_of_climb_prop`

The maximum rate of climb of a propeller aircraft (constant shaft power and propeller efficiency) at an altitude, the speed for it (minimum power required) and the climb angle there.

$$RC_{max} = \frac{\eta_p P - P_{R,min}}{W}, \quad V_{mp} = \sqrt{\frac{2W}{\rho S}} \left(\frac{K}{3C_{D0}} \right)^{1/4}$$

input	meaning	condition
`W`	weight	

input	meaning	condition
`S`	wing area	
`h`	altitude	
`P`	shaft power (W, kW, hp)	
`eta`	propeller efficiency	
`CD0`	zero-lift drag coefficient	
`AR`	aspect ratio	
`e`	Oswald efficiency factor	

output	meaning
`RC_max`	maximum rate of climb
`V`	speed for the maximum rate of climb
`gamma_deg`	climb angle in degrees

- **Method:** RC = (eta P - D V)/W is largest at the minimum-power speed $V = sqrt$(2W/(rho S)) (K/(3 CD0))^$(1/4)$
- **Assumes:** available power above the minimum power required; climb angle below 30 degrees (lift = weight); parabolic drag polar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_max_rate_of_climb_prop', W=11000.0, S=16.0, h=0.0,
P=120000.0, eta=0.8, CD0=0.03, AR=7.5, e=0.8)
# -> RC_max = 6.02265 m/s, V = 29.356 m/s, gamma_deg = 11.8388   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum rate of climb of a propeller aircraft: W = 11 kN, S = 16 m^2, h = 0 m, P = 1 kW, eta = 0.8, CD0 = 0.03, AR = 7.5, e = 0.8	INVALID_INPUT	Maximum rate of climb of a propeller aircraft: the available power is below the minimum power required.
Maximum rate of climb of a propeller aircraft: W = 11 kN, S = 16 m^2, h = 0 m, eta = 0.8, CD0 = 0.03, AR = 7.5, e = 0.8	NEEDS_INPUT	aero_max_rate_of_climb_prop needs P (shaft power (W, kW, hp)). Example: solve('aero_max_rate_of_climb_prop', W='11000 N'

Absolute ceiling of a jet aircraft — `aero_absolute_ceiling_jet`

The absolute ceiling of a jet aircraft (where the available thrust just equals the minimum drag) when the thrust falls in proportion to the air density, $T = T_0$ $\rho/rho0$.

$$\sigma_{abs} = \frac{2W\sqrt{KC_{D0}}}{T_0} = \frac{W}{T_0 (L/D)_{max}}$$

input	meaning	condition
`W`	weight	

input	meaning	condition
`S`	wing area	
`T0`	sea-level thrust	
`CD0`	zero-lift drag coefficient	
`AR`	aspect ratio	
`e`	Oswald efficiency factor	

output	meaning
`h_abs`	absolute ceiling
`h_abs_ft`	absolute ceiling in feet
`sigma`	density ratio at the ceiling

- **Method:** T0 sigma = D_min = 2W sqrt(K CD0): $\sigma = 2W$ sqrt(K CD0)/T0; the height of that density in the standard atmosphere
- **Assumes:** minimum drag below the sea-level thrust; ceiling below 47 km; thrust proportional to density (stated lapse); parabolic drag polar
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_absolute_ceiling_jet', W=100000.0, S=30.0,
T0=13486.71062689471, CD0=0.02, AR=7.0, e=0.8)
# -> h_abs = 6669.77 m, h_abs_ft = 21882.4, sigma = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Absolute ceiling: W = 100 kN, S = 30 m^2, T0 = 1 kN, CD0 = 0.02, AR = 7, e = 0.8	INVALID_INPUT	Absolute ceiling of a jet aircraft: the sea-level thrust is below the minimum drag: the aircraft cannot fly level even a
Absolute ceiling: W = 100 kN, S = 30 m^2, CD0 = 0.02, AR = 7, e = 0.8	NEEDS_INPUT	aero_absolute_ceiling_jet needs T0 (sea-level thrust). Example: solve('aero_absolute_ceiling_jet', W='100000 N', S='30 m

Service ceiling of a jet aircraft — `aero_service_ceiling_jet`

The service ceiling of a jet aircraft: the altitude where its maximum rate of climb has fallen to 100 ft/min (0.508 m/s, or a stated value), with thrust proportional to density.

$$RC_{max}(h_s) = RC_s, \quad T(h) = T_0 \frac{\rho(h)}{\rho_0}$$

input	meaning	condition
`W`	weight	
`S`	wing area	
`T0`	sea-level thrust	
`CD0`	zero-lift drag coefficient	
`AR`	aspect ratio	

input	meaning	condition
`e`	Oswald efficiency factor	
`RCs`	rate of climb that defines the ceiling (default 100 ft/min = 0.508 m/s)	

output	meaning
`h_s`	service ceiling
`h_s_ft`	service ceiling in feet

- **Method:** solve RC_max(h) = RCs by bisection, with RC_max from the optimum climb speed V^2 = (T + sqrt(T^2 + 12ab))/(6a) and $T = T_0\ \rho/rho0$
- **Assumes:** $T_0 < 0.8$ W; climb possible at sea level; ceiling below 47 km; thrust proportional to density; small climb angle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_service_ceiling_jet', W=100000.0, S=30.0, T0=30000.0,
CD0=0.02, AR=7.0, e=0.8)
# -> h_s = 12557.9 m, h_s_ft = 41200.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Service ceiling: W = 100 kN, S = 30 m^2, T0 = 1 kN, CD0 = 0.02, AR = 7, e = 0.8	INVALID_INPUT	Service ceiling of a jet aircraft: the aircraft cannot climb at the service-ceiling rate even at sea level.
Service ceiling: W = 100 kN, S = 30 m^2, CD0 = 0.02, AR = 7, e = 0.8	NEEDS_INPUT	aero_service_ceiling_jet needs T0 (sea-level thrust). Example: solve('aero_service_ceiling_jet', W='100000 N', S='30 m^2

Neutral point and static margin (wing and tail) — `aero_neutral_point`

The stick-fixed neutral point, the static margin, the airplane lift-curve slope and the pitching-moment slope Cm_alpha of a wing-tail aircraft (positions as fractions of the mean aerodynamic chord, aft of its leading edge).

$$h_n = h_{ac} + \eta\,\bar{V}_H\,\frac{a_t}{a}\left(1 - \frac{d\varepsilon}{d\alpha}\right), \quad \bar{V}_H = \frac{S_t l_t}{S\bar{c}}, \quad C_{m\alpha} = -a\,(h_n - h_{cg})$$

input	meaning	condition
`h_ac`	wing-body aerodynamic centre / mean chord	
`a_w`	wing lift-curve slope (per rad)	
`a_t`	tail lift-curve slope (per rad)	
`St_S`	tail area / wing area	
`lt_c`	distance from the wing aerodynamic centre to the tail aerodynamic centre / mean chord	
`deps`	downwash gradient d(epsilon)/d(alpha)	
`h_cg`	centre of gravity / mean chord	
`eta_t`	tail efficiency (dynamic-pressure ratio, default 1)	

output	meaning
`h_n`	neutral point / mean chord
`SM`	static margin h_n - h_cg (positive = stable)
`CL_alpha`	airplane lift-curve slope (per rad)
`Cm_alpha`	pitching-moment slope about the cg (per rad)

- **Method:** $a = a_w + \eta$ (St/S) a_t (1 - deps); $h_n = h_a c + \eta$ (St/S)(lt/c) a_t (1 - deps)/a; $SM = h_n - h_c g$; $Cm_a lpha = -a$ SM
- **Assumes:** linear aerodynamics (small angles); thrust and fuselage moment effects neglected beyond h_ac; stated ranges of each input
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_neutral_point', h_ac=0.25, a_w=5.0, a_t=5.0, St_S=0.2,
lt_c=4.0, deps=0.5, h_cg=0.3)
# -> h_n = 0.613636, SM = 0.313636, CL_alpha = 5.5, Cm_alpha = -1.725   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Neutral point: h_ac = 0.25, a_w = 5, a_t = 5, St_S = 0.2, lt_c = 4, deps = 0.5, h_cg = 0.3	SOLVED	h_n = 0.613636, SM = 0.313636, CL_alpha = 5.5, Cm_alpha = -1.725
Neutral point: h_ac = 0.25, a_w = 5, a_t = 5, St_S = 0.2, lt_c = 4, deps = 1.5, h_cg = 0.3	INVALID_INPUT	Neutral point and static margin (wing and tail): the downwash gradient must lie between 0 and 0.9.
Neutral point: h_ac = 0.25, a_w = 5, a_t = 5, St_S = 0.2, lt_c = 4, deps = 0.5	NEEDS_INPUT	aero_neutral_point needs h_cg (centre of gravity / mean chord). Example: solve('aero_neutral_point', h_ac=0.25, a_w=5, a

Ideal hover power of a rotor (momentum theory) — `aero_actuator_disk_hover`

The induced velocity at the disk, the ideal (induced) power and the far-wake velocity of a hovering rotor or static propeller from its thrust, diameter and the air density (actuator-disk momentum theory).

$$v_i = \sqrt{\frac{T}{2\rho A}}, \quad P_i = T v_i = \frac{T^{3/2}}{\sqrt{2\rho A}}$$

input	meaning	condition
`T`	thrust	
`D`	rotor (disk) diameter	
`rho`	air density	

output	meaning
`v_i`	induced velocity at the disk
`P_ideal`	ideal induced power
`w`	far-wake velocity

- **Method:** $v_i = sqrt$(T/(2 rho A)), $A = pi\ D^2/4$; $P = T$ v_i; $w = 2$ v_i
- **Assumes:** T, D, $\rho > 0$; ideal actuator disk: uniform, inviscid, no swirl
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_actuator_disk_hover', T=6.283185307179586, D=2.0, rho=1.0)
# -> v_i = 1 m/s, P_ideal = 6.28319 W, w = 2 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hover power: T = 6.283185307179586 N, D = 2 m, rho = 1 kg/m^3	SOLVED	v_i = 1 m/s, P_ideal = 6.28319 W, w = 2 m/s
Hover power: T = 20 kN, D = 0 m, rho = 1.225 kg/m^3	INVALID_INPUT	Ideal hover power of a rotor (momentum theory): the thrust, the disk diameter and the air density must be a positive num
Hover power: T = 20 kN, D = 10 m	NEEDS_INPUT	aero_actuator_disk_hover needs rho (air density). Example: solve('aero_actuator_disk_hover', T='6.28319 N', D='2 m', rho

Ideal propeller in axial flight (momentum theory) — `aero_actuator_disk_forward`

The induced velocity, ideal power, ideal (Froude) propulsive efficiency and far-wake velocity increment of a propeller producing a thrust at a flight speed (actuator-disk momentum theory).

$$v_i = -\frac{V}{2} + \sqrt{\frac{V^2}{4} + \frac{T}{2\rho A}}, \quad \eta_i = \frac{V}{V + v_i}$$

input	meaning	condition
`T`	thrust	
`V`	flight (axial) speed	
`D`	propeller diameter	
`rho`	air density	

output	meaning
`v_i`	induced velocity at the disk
`P_ideal`	ideal power absorbed
`eta_ideal`	ideal propulsive efficiency T V/P
`w`	far-wake velocity increment

- **Method:** $v = -V/2 + sqrt$(V^2/4 + T/(2 rho A)); $P = T$ (V + v); $\eta = V/(V + v)$; $w = 2v$
- **Assumes:** T, V, D, $\rho > 0$; ideal actuator disk
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_actuator_disk_forward', T=25.132741228718345, V=3.0,
D=2.0, rho=1.0)
# -> v_i = 1 m/s, P_ideal = 100.531 W, eta_ideal = 0.75, w = 2 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ideal propeller: T = 25.132741228718345 N, V = 3 m/s, D = 2 m, rho = 1 kg/m^3	SOLVED	v_i = 1 m/s, P_ideal = 100.531 W, eta_ideal = 0.75, w = 2 m/s
Ideal propeller: T = 3 kN, V = -60 m/s, D = 2 m, rho = 1.225 kg/m^3	INVALID_INPUT	Ideal propeller in axial flight (momentum theory): the flight speed must be a positive number.
Ideal propeller: T = 3 kN, V = 60 m/s, D = 2 m	NEEDS_INPUT	aero_actuator_disk_forward needs rho (air density). Example: solve('aero_actuator_disk_forward', T='25.1327 N', V='3 m/s

Thin-airfoil theory for a NACA 4-digit airfoil — `aero_naca4_thin_airfoil`

The zero-lift angle, lift coefficient and the pitching-moment coefficients about the quarter chord and the leading edge of a cambered NACA 4-digit airfoil at an angle of attack, from thin-airfoil theory.

$$C_L = 2\pi(\alpha - \alpha_{L0}), \quad \alpha_{L0} = -\frac{1}{\pi}\int_0^{\pi} \frac{dz}{dx}(\cos\theta - 1)\, d\theta, \quad C_{m,c/4} = \frac{\pi}{4}(A_2 - A_1)$$

input	meaning	condition
`m`	maximum camber as a fraction of the chord (first digit / 100)	
`p`	chordwise position of maximum camber (second digit / 10; 0 for a symmetric section)	
`alpha`	angle of attack (deg)	

output	meaning
`alpha_L0_deg`	zero-lift angle of attack in degrees
`CL`	section lift coefficient
`Cm_c4`	moment coefficient about the quarter chord
`Cm_le`	moment coefficient about the leading edge

- **Method:** Glauert: $A_0 = \alpha - (1/pi)$ int dz/dx dtheta, $An = (2/pi)$ int dz/dx cos(n theta) dtheta; $CL = pi$(2A0 + A1); $Cm_c/4 = pi/4\ (A_2 - A_1)$; integrals of the NACA mean line in closed form
- **Assumes:** $0 \le m \le 0.095$; $0.05 \le p \le 0.95$ when $m > 0$; |alpha| <= 15 deg; thin airfoil, inviscid, incompressible, unstalled
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_naca4_thin_airfoil', m=0.0, p=0.4, alpha=0.1)
# -> alpha_L0_deg = 0, CL = 0.628319, Cm_c4 = 0, Cm_le = -0.15708   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
NACA 0012 thin airfoil: alpha = 0.1 rad	SOLVED	alpha_L0_deg = 0, CL = 0.628319, Cm_c4 = 0, Cm_le = -0.15708
NACA 2412 thin airfoil: alpha = 30 deg	INVALID_INPUT	Thin-airfoil theory for a NACA 4-digit airfoil: the angle of attack must lie within +-15 degrees (thin-airfoil theory, u

question	outcome	result
NACA 2412 thin airfoil:	NEEDS_INPUT	aero_naca4_thin_airfoil needs alpha (angle of attack (deg)). Example: solve('aero_naca4_thin_airfoil', m=0, p=0.4, alpha

Elliptic wing by lifting-line theory — `aero_elliptic_wing_lifting_line`

The lift-curve slope, lift coefficient, induced drag coefficient and induced angle of attack of an untwisted elliptic wing of a given aspect ratio at an angle of attack measured from zero lift (Prandtl lifting-line theory, span efficiency 1).

$$a = \frac{a_0}{1 + \frac{a_0}{\pi AR}}, \quad C_{D_i} = \frac{C_L^2}{\pi AR}, \quad \alpha_i = \frac{C_L}{\pi AR}$$

input	meaning	condition
`AR`	aspect ratio	
`alpha`	angle of attack from zero lift (deg)	
`a0`	section lift-curve slope per radian (default 2 pi)	

output	meaning
`CL_alpha`	wing lift-curve slope per radian
`CL`	wing lift coefficient
`CDi`	induced drag coefficient
`alpha_i_deg`	induced angle of attack in degrees

- **Method:** a = a0/(1 + a0/(pi AR)); $CL = a$ alpha; CDi = CL^2/(pi AR); alpha_i = CL/(pi AR)
- **Assumes:** $2 \le AR \le 40$; $3 \le a_0 \le 7.5$ per rad; |alpha| <= 15 deg; untwisted elliptic planform, attached flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_elliptic_wing_lifting_line', AR=2.0, alpha=0.1)
# -> CL_alpha = 3.14159, CL = 0.314159, CDi = 0.015708, alpha_i_deg = 2.86479   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Elliptic wing: AR = 2, alpha = 0.1 rad	SOLVED	CL_alpha = 3.14159, CL = 0.314159, CDi = 0.015708, alpha_i_deg = 2.86479
Elliptic wing: AR = 1, alpha = 5 deg	INVALID_INPUT	Elliptic wing by lifting-line theory: the aspect ratio must lie between 2 and 40 (lifting-line theory).
Elliptic wing: AR = 8	NEEDS_INPUT	aero_elliptic_wing_lifting_line needs alpha (angle of attack from zero lift (deg)). Example: solve('aero_elliptic_wing_l

Wind triangle: heading and ground speed — `aero_wind_triangle`

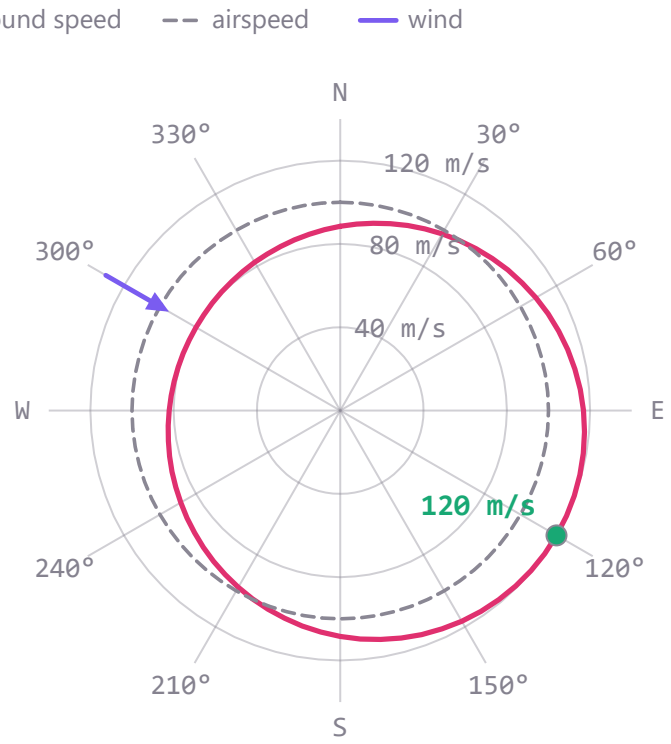
The heading to fly, the wind correction angle and the ground speed for an aircraft of a given true airspeed to make good a true course in a wind (direction the wind blows from, and its speed).

$$\sin\delta = \frac{V_w \sin(W - C)}{V}, \quad V_g = V\cos\delta - V_w \cos(W - C)$$

input	meaning	condition
`TAS`	true airspeed (kt, km/h, m/s)	
`course`	true course to make good (deg)	
`wind_from`	direction the wind blows from (deg true)	
`wind_speed`	wind speed	

output	meaning
`heading_deg`	true heading to fly, degrees
`wca_deg`	wind correction angle, degrees (positive = right of course)
`GS`	ground speed
`GS_kt`	ground speed in knots

- **Method:** WCA = asin(Vw sin(W - C)/TAS); heading = C + WCA; GS = TAS cos(WCA) - Vw cos(W - C)
- **Assumes:** crosswind component below 0.99 TAS; positive ground speed; angles in [0, 360] degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ground speed by course [m/s] (north up)

Ground speed for every course (5° steps) of an aircraft flying 100 m/s through a 20 m/s wind from 300°, each solved by the package's wind triangle: slowest 80 m/s heading into the wind (course 300°), fastest 120 m/s with it (course 120°, marked); largest wind correction 11.537°.

```
aerospace_engineering_ai999x.solve('aero_wind_triangle', TAS=100.0, course=1.5707963267948966,
wind_from=1.5707963267948966, wind_speed=20.0)
# -> heading_deg = 90, wca_deg = 0, GS = 80 m/s, GS_kt = 155.508   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_wind_triangle', TAS=100.0, course=0.5,
wind_from=2.0707963267948966, wind_speed=50.0)
# -> heading_deg = 58.6479, wca_deg = 30, GS = 86.6025 m/s, GS_kt = 168.342   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wind triangle: TAS = 100 kt, course = 90 deg, wind_from = 90 deg, wind_speed = 20 kt	SOLVED	heading_deg = 90, wca_deg = 0, GS = 41.1556 m/s, GS_kt = 80
Wind triangle: TAS = 100 kt, course = 90 deg, wind_from = 0 deg, wind_speed = 120 kt	INVALID_INPUT	Wind triangle: heading and ground speed: the crosswind is too strong for this airspeed (no heading holds the course).
Wind triangle: TAS = 100 kt, course = 90 deg, wind_from = 0 deg	NEEDS_INPUT	aero_wind_triangle needs wind_speed (wind speed). Example: solve('aero_wind_triangle', TAS='100 m/s', course='1.5708 rad

Total (stagnation) air temperature in flight — `aero_total_temperature`

The total (stagnation) air temperature, the ram temperature rise and the Mach number of an aircraft flying at a true airspeed through air of a static temperature (full recovery, perfect gas, air gamma = 1.4).

$$T_0 = T + \frac{V^2}{2c_p} = T\left(1 + \frac{\gamma - 1}{2}M^2\right)$$

input	meaning	condition
`V`	true airspeed	
`T`	static (outside) air temperature	

output	meaning
`T0`	total (stagnation) temperature
`dT`	ram temperature rise
`M`	flight Mach number

- **Method:** T0 = T + V^2/(2 cp), $cp = \gamma R/(\gamma - 1) = 1004.69$ J/(kg K); $M = V/sqrt$(gamma R T)
- **Assumes:** 150 $K \le T \le 350$ K; $M < 5$; full recovery (recovery factor 1)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_total_temperature', V=316.96778261595074, T=250.0)
# -> T0 = 300 K, dT = 50 K, M = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Total temperature: V = 316.96778261595074 m/s, T = 250 K	SOLVED	T0 = 300 K, dT = 50 K, M = 1
Total temperature: V = 250 m/s, T = 500 K	INVALID_INPUT	Total (stagnation) air temperature in flight: the static air temperature must lie between 150 K and 350 K.
Total temperature: V = 250 m/s	NEEDS_INPUT	aero_total_temperature needs T (static (outside) air temperature). Example: solve('aero_total_temperature', V='316.968 m

Optimal staging of a multistage rocket — `aero_optimal_staging`

The lightest rocket that gives a payload a required delta-v with stages of given specific impulses and structural coefficients: the optimal mass ratio and mass of each stage, the lift-off mass and the payload fraction (Lagrange-multiplier solution).

$$R_i = \frac{\lambda c_i - 1}{\lambda c_i \varepsilon_i}, \quad \sum_i c_i \ln R_i = \Delta v, \quad m_i = m_{above}\frac{R_i - 1}{1 - R_i \varepsilon_i}$$

input	meaning	condition
`Isp`	specific impulse of each stage in seconds, bottom stage first, e.g. '[300, 350, 450]'	
`eps`	structural coefficient ms/(ms + mp) of each stage	
`dv`	required total delta-v	
`m_pl`	payload mass	

output	meaning
`m0`	minimum lift-off mass
`payload_fraction`	payload fraction m_pl/m_0
`R`	optimal mass ratio of each stage (list)
`stage_mass`	mass of each stage, structure plus propellant, in kg (list)

- **Method:** R_i = (lambda $c_i - 1)/(\lambda$ c_i eps_i) with the multiplier lambda from sum c_i ln $R_i = dv$ (bisection); stage masses built from the payload up, m_i = m_above (R_i - 1)/(1 - R_i eps_i)
- **Assumes:** 2 to 5 stages; 0.01 <= eps <= 0.5; delta-v below what the stages could reach with zero payload; impulsive staging, no gravity or drag losses
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_optimal_staging', Isp='300, 300', eps='0.1, 0.1',
dv=8156.942157885825, m_pl=1000.0)
# -> m0 = 36000 kg, payload_fraction = 0.0277778, R = [4.0, 4.0], stage_mass = [30000.0, 5000.0]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Optimal staging: Isp = [300, 300], eps = [0.1, 0.1], dv = 8156.942157885825 m/s, m_pl = 1000 kg	SOLVED	m0 = 36000 kg, payload_fraction = 0.0277778, R = [4.0, 4.0], stage_mass = [30000.0, 5000.0]

question	outcome	result
Optimal staging: Isp = [300, 300], eps = [0.1, 0.1], dv = 20 km/s, m_pl = 1000 kg	INVALID_INPUT	Optimal staging of a multistage rocket: the delta-v is not reachable with these stages.
Optimal staging: Isp = [300, 300], eps = [0.1, 0.1], dv = 9 km/s	NEEDS_INPUT	aero_optimal_staging needs m_pl (payload mass). Example: solve('aero_optimal_staging', Isp='300, 300', eps='0.1, 0.1', d

Coverage of a satellite at an altitude — `aero_satellite_coverage`

The Earth central angle out to the edge of a satellite's coverage, the off-nadir angle and slant range to that edge, and the fraction of the Earth's surface in view, for a minimum elevation angle at the ground (0 = the geometric horizon).

$$\sin\eta = \frac{R\cos\varepsilon}{R+h}, \quad \lambda = 90^\circ - \varepsilon - \eta, \quad d = \frac{R\sin\lambda}{\sin\eta}, \quad f = \frac{1-\cos\lambda}{2}$$

input	meaning	condition
`h`	satellite altitude	
`eps_min`	minimum elevation angle at the ground station (default 0 deg)	

output	meaning
`lambda_deg`	Earth central angle to the coverage edge, degrees
`nadir_deg`	off-nadir angle to the edge, degrees
`slant_range`	slant range to the coverage edge
`coverage`	fraction of the Earth's surface covered

- **Method:** sin(eta) = R cos(eps)/(R + h); $\lambda = 90$ deg - eps - eta; $d = R$ sin(lambda)/sin(eta); coverage = (1 - cos(lambda))/2
- **Assumes:** 100 $km \le h \le 1e6$ km; 0 <= eps_min <= 80 deg; spherical Earth of radius 6378.1366 km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_satellite_coverage', h=6378136.6)
# -> lambda_deg = 60, nadir_deg = 30, slant_range = 1.10473e+07 m, coverage = 0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Satellite coverage: h = 6378.1366 km	SOLVED	lambda_deg = 60, nadir_deg = 30, slant_range = 1.10473e+07 m, coverage = 0.25
Satellite coverage: h = 800 km, eps_min = 85 deg	INVALID_INPUT	Coverage of a satellite at an altitude: the minimum elevation angle must lie between 0 and 80 degrees.
Satellite coverage:	NEEDS_INPUT	aero_satellite_coverage needs h (satellite altitude). Example: solve('aero_satellite_coverage', h='6.37814e+06 m').

Stall speed in level flight — `aero_stall_speed`

The stall speed $V_s = sqrt$(2W/(rho S CL_max)) of an aircraft in level flight at an altitude of the standard atmosphere, as true and equivalent airspeed.

$$V_s = \sqrt{\frac{2W}{\rho S C_{L,max}}}$$

input	meaning	condition
`W`	aircraft weight (N, kN, lbf)	
`S`	wing area (m^2, ft^2)	
`CLmax`	maximum lift coefficient	
`h`	altitude (standard atmosphere)	

output	meaning
`V_stall`	stall speed (true airspeed)
`V_stall_kt`	stall speed in knots (TAS)
`EAS_stall`	stall speed as equivalent airspeed
`wing_loading`	wing loading W/S

- **Method:** $V_s = sqrt$(2 W/(rho S CL_max)); EAS = V sqrt(rho/rho0)
- **Assumes:** W, $S > 0$; 0.5 <= CL_max <= 5; -1 $km \leq h \leq 47$ km; level flight, load factor 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_stall_speed', W=10000.0, S=16.0, CLmax=1.6, h=0.0)
# -> V_stall = 25.2538 m/s, V_stall_kt = 49.0895, EAS_stall = 25.2538 m/s, wing_loading = 625 N/m^2
(SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_stall_speed', W=600000.0, S=120.0, CLmax=2.5, h=3000.0)
# -> V_stall = 66.3265 m/s, V_stall_kt = 128.928, EAS_stall = 57.1429 m/s, wing_loading = 5000
N/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stall speed: W = 10 kN, S = 16 m^2, CLmax = 1.6, h = 0 m	SOLVED	V_stall = 25.2538 m/s, V_stall_kt = 49.0895, EAS_stall = 25.2538 m/s, wing_loading = 625 N/m^2
Stall speed: W = 10 kN, S = 16 m^2, CLmax = 9, h = 0 m	INVALID_INPUT	Stall speed in level flight: the maximum lift coefficient must lie between 0.5 and 5.
Stall speed: W = 10 kN, S = 16 m^2, CLmax = 1.6	NEEDS_INPUT	aero_stall_speed needs h (altitude (standard atmosphere)). Example: solve('aero_stall_speed', W='10000 N', S='16 m^2', C

Lift coefficient in level flight — `aero_level_flight_lift_coefficient`

The lift coefficient CL = 2W/(rho V^2 S) needed for steady level flight at a true airspeed and altitude, with the dynamic pressure.

$$C_L = \frac{2W}{\rho V^2 S}$$

input	meaning	condition
`W`	aircraft weight	
`S`	wing area	
`V`	true airspeed (m/s, kt, km/h)	
`h`	altitude (standard atmosphere)	

output	meaning
`CL`	lift coefficient
`q`	dynamic pressure
`wing_loading`	wing loading W/S

- **Method:** CL = W/(q S), $q = \rho V^2/2$
- **Assumes:** W, S, $V > 0$; -1 $km \le h \le 47$ km; level flight, $L = W$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_level_flight_lift_coefficient', W=10000.0, S=16.0, V=50.0, h=0.0)
# -> CL = 0.408164, q = 1531.25 Pa, wing_loading = 625 N/m^2   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_level_flight_lift_coefficient', W=600000.0, S=120.0, V=230.0, h=11000.0)
# -> CL = 0.518188, q = 9649 Pa, wing_loading = 5000 N/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lift coefficient in level flight: W = 10 kN, S = 16 m^2, V = 50 m/s, h = 0 m	SOLVED	CL = 0.408164, q = 1531.25 Pa, wing_loading = 625 N/m^2
Lift coefficient in level flight: W = 10 kN, S = 16 m^2, V = 50 m/s, h = 80 km	INVALID_INPUT	Lift coefficient in level flight: the altitude must lie between -1 km and 47 km (standard atmosphere layers to 47 km).
Lift coefficient in level flight: W = 10 kN, S = 16 m^2, V = 50 m/s	NEEDS_INPUT	aero_level_flight_lift_coefficient needs h (altitude (standard atmosphere)). Example: solve('aero_level_flight_lift_coef

Wing loading — `aero_wing_loading`

The wing loading W/S of an aircraft in N/m^2, as a mass per area (kg/m^2) and in lbf/ft^2.

$$\frac{W}{S}$$

input	meaning	condition
`W`	aircraft weight (N, kN, lbf)	

input	meaning	condition
`S`	wing area (m^2, ft^2)	

output	meaning
`wing_loading`	wing loading W/S
`wing_loading_kg_m2`	wing loading as mass per area, kg/m^2
`wing_loading_psf`	wing loading in lbf/ft^2

- **Method:** W/S; W/(g0 S); 1 lbf/ft^2 = 47.880 N/m^2
- **Assumes:** W, $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_wing_loading', W=600000.0, S=120.0)
# -> wing_loading = 5000 N/m^2, wing_loading_kg_m2 = 509.858, wing_loading_psf = 104.427   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_wing_loading', W=10675.7318766252, S=14.8644864)
# -> wing_loading = 718.204 N/m^2, wing_loading_kg_m2 = 73.2364, wing_loading_psf = 15   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wing loading: W = 600 kN, S = 120 m^2	SOLVED	wing_loading = 5000 N/m^2, wing_loading_kg_m2 = 509.858, wing_loading_psf = 104.427
Wing loading: W = 2400 lbf, S = 160 ft^2	SOLVED	wing_loading = 718.204 N/m^2, wing_loading_kg_m2 = 73.2364, wing_loading_psf = 15
Wing loading: W = 600 kN, S = 0 m^2	INVALID_INPUT	Wing loading: the weight and the wing area must be a positive number.
Wing loading: W = 600 kN	NEEDS_INPUT	aero_wing_loading needs S (wing area (m^2, ft^2)). Example: solve('aero_wing_loading', W='600000 N', S='120 m^2').

Induced drag in level flight — `aero_induced_drag`

The lift coefficient, induced drag coefficient CL^2/(pi e AR), induced drag and induced power of an aircraft in level flight at a true airspeed and altitude.

$$C_{D_i} = \frac{C_L^2}{\pi e AR}, \quad D_i = \frac{W^2}{qS\,\pi e AR}$$

input	meaning	condition
`W`	aircraft weight	
`S`	wing area	
`V`	true airspeed	
`h`	altitude (standard atmosphere)	
`AR`	wing aspect ratio b^2/S	
`e`	Oswald span efficiency factor	

output	meaning
`CL`	lift coefficient
`CDi`	induced drag coefficient
`Di`	induced drag
`P_induced`	induced power Di V

- **Method:** CL = W/(q S), CDi = CL^2/(pi e AR), $Di = q$ S CDi
- **Assumes:** W, S, $V > 0$; $1 \le AR \le 50$; $0.3 \le e \le 1$; level flight, $L = W$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_induced_drag', W=10000.0, S=16.0, V=50.0, h=0.0, AR=8.0,
e=0.8)
# -> CL = 0.408164, CDi = 0.00828588, Di = 203.004 N, P_induced = 10150.2 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Induced drag: W = 10 kN, S = 16 m^2, V = 50 m/s, h = 0 m, AR = 8, e = 0.8	SOLVED	CL = 0.408164, CDi = 0.00828588, Di = 203.004 N, P_induced = 10150.2 W
Induced drag: W = 10 kN, S = 16 m^2, V = 50 m/s, h = 0 m, AR = 8, e = 1.4	INVALID_INPUT	Induced drag in level flight: the Oswald efficiency factor must lie between 0.3 and 1.
Induced drag: W = 10 kN, S = 16 m^2, V = 50 m/s, h = 0 m, AR = 8	NEEDS_INPUT	aero_induced_drag needs e (Oswald span efficiency factor). Example: solve('aero_induced_drag', W='10000 N', S='16 m^2',

Glide ratio, angle and range — `aero_glide_range`

The still-air glide range $R = h$ (L/D) from a height h, the glide angle atan(D/L) and, with the airspeed, the sink rate and time aloft.

$$R = h\,\frac{L}{D}, \quad \tan\gamma = \frac{D}{L}$$

input	meaning	condition
`dh`	height to be lost	
`LD`	lift-to-drag (glide) ratio	
`V`	airspeed along the glide path (optional)	

output	meaning
`range`	glide range over the ground
`range_km`	glide range, km
`glide_angle_deg`	glide angle below the horizon, degrees
`sink_rate`	sink rate V sin(gamma)
`time`	time to lose the height

- **Method:** tan(gamma) = D/L; $R = h$ L/D; sink rate = V sin(gamma)
- **Assumes:** $h > 0$; $0 < L/D \leq 100$; still air, steady glide
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_glide_range', dh=2000.0, LD=15.0)
# -> range = 30000 m, range_km = 30, glide_angle_deg = 3.81407   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_glide_range', dh=1000.0, LD=40.0, V=25.0)
# -> range = 40000 m, range_km = 40, glide_angle_deg = 1.4321, sink_rate = 0.624805 m/s, time =
1600.5 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Glide range: h = 2000 m, LD = 15	SOLVED	range = 30000 m, range_km = 30, glide_angle_deg = 3.81407
Glide range: h = 1000 m, LD = 40, V = 25 m/s	SOLVED	range = 40000 m, range_km = 40, glide_angle_deg = 1.4321, sink_rate = 0.624805 m/s, time = 1600.5 s
Glide range: h = 2000 m, LD = -15	INVALID_INPUT	Glide ratio, angle and range: the height and the lift-to-drag ratio must be a positive number.
Glide range: h = 2000 m	NEEDS_INPUT	aero_glide_range needs LD (lift-to-drag (glide) ratio). Example: solve('aero_glide_range', dh='2000 m', LD=15).

Rocket thrust and specific impulse from mass flow and exhaust velocity —

`aero_rocket_thrust`

The thrust F = mdot ve + (pe - pa) Ae of a rocket engine, its specific impulse F/(mdot g0) and effective exhaust velocity, from the propellant mass flow and the exhaust velocity (with the pressure thrust when the exit conditions are given).

$$F = \dot{m} v_e + (p_e - p_a) A_e, \quad I_{sp} = \frac{F}{\dot{m} g_0}$$

input	meaning	condition
`mdot`	propellant mass flow rate (kg/s)	
`ve`	exhaust velocity at the nozzle exit	
`pe`	exit pressure (optional, with pa and Ae)	
`pa`	ambient pressure (optional)	
`Ae`	nozzle exit area (optional)	

output	meaning
`F`	thrust
`F_kN`	thrust, kN
`Isp`	specific impulse F/(mdot g0)
`c_eff`	effective exhaust velocity F/mdot

output	meaning
`F_pressure`	pressure thrust (pe - pa) Ae

- **Method:** F = mdot ve + (pe - pa) Ae; Isp = F/(mdot g0); c = F/mdot
- **Assumes:** mdot, $ve > 0$; pe, pa, Ae given together or not at all; steady, one-dimensional exit flow
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_rocket_thrust', mdot=250.0, ve=3000.0)
# -> F = 750000 N, F_kN = 750, Isp = 305.915 s, c_eff = 3000 m/s, F_pressure = 0 N   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_rocket_thrust', mdot=250.0, ve=3000.0, pe=70000.0,
pa=101325.0, Ae=1.2)
# -> F = 712410 N, F_kN = 712.41, Isp = 290.582 s, c_eff = 2849.64 m/s, F_pressure = -37590 N
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rocket thrust: mdot = 250 kg/s, ve = 3000 m/s	SOLVED	F = 750000 N, F_kN = 750, Isp = 305.915 s, c_eff = 3000 m/s, F_pressure = 0 N
Rocket thrust: mdot = 250 kg/s, ve = 3000 m/s, pe = 70 kPa	INVALID_INPUT	Rocket thrust and specific impulse from mass flow and exhaust velocity: give the exit pressure, ambient pressure and exi
Rocket thrust: mdot = 250 kg/s	NEEDS_INPUT	aero_rocket_thrust needs ve (exhaust velocity at the nozzle exit). Example: solve('aero_rocket_thrust', mdot='250 kg/s',

Ballistic coefficient — `aero_ballistic_coefficient`

The ballistic coefficient beta = m/(CD A) of a body (mass per unit drag area), in kg/m^2 and as a weight per area in lbf/ft^2.

$$\beta = \frac{m}{C_D A}$$

input	meaning	condition
`m`	mass	
`CD`	drag coefficient	
`A`	reference area	

output	meaning
`beta`	ballistic coefficient m/(CD A)
`beta_psf`	ballistic coefficient as weight per drag area, lbf/ft^2

- **Method:** beta = m/(CD A)
- **Assumes:** m, CD, $A > 0$; $CD \leq 5$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_ballistic_coefficient', m=1000.0, CD=1.2, A=1.5)
# -> beta = 555.556 kg/m^2, beta_psf = 113.787   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_ballistic_coefficient', m=50.0, CD=2.0, A=0.25)
# -> beta = 100 kg/m^2, beta_psf = 20.4816   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Ballistic coefficient: m = 1000 kg, CD = 1.2, A = 1.5 m^2	SOLVED	beta = 555.556 kg/m^2, beta_psf = 113.787
Ballistic coefficient: m = 1000 kg, CD = -1.2, A = 1.5 m^2	INVALID_INPUT	Ballistic coefficient: the mass, drag coefficient and reference area must be a positive number.
Ballistic coefficient: m = 1000 kg, CD = 1.2	NEEDS_INPUT	aero_ballistic_coefficient needs A (reference area). Example: solve('aero_ballistic_coefficient', m='1000 kg', CD=1.2, A

Mach number, dynamic pressure, equivalent airspeed and Reynolds number at an altitude — `aero_flight_condition`

For a true airspeed at an altitude of the standard atmosphere: the Mach number, dynamic pressure, equivalent airspeed and (with a reference length) the Reynolds number.

$$M = \frac{V}{\sqrt{\gamma R T}}, \quad q = \tfrac{1}{2}\rho V^2, \quad V_E = V\sqrt{\rho/\rho_0}, \quad Re = \frac{\rho V L}{\mu}$$

input	meaning	condition
`V`	true airspeed (m/s, kt, km/h)	
`h`	geometric altitude (standard atmosphere)	
`L`	reference length for the Reynolds number, e.g. the chord (optional)	

output	meaning
`M`	Mach number
`q`	dynamic pressure
`EAS`	equivalent airspeed
`EAS_kt`	equivalent airspeed, knots
`a`	speed of sound
`rho`	air density
`Re`	Reynolds number rho V L/mu (with L)

- **Method:** $M = V/a$, $a = sqrt$(gamma R T); $q = \rho\, V^2/2$; EAS = V sqrt(rho/rho0); $Re = \rho$ V L/mu (Sutherland)
- **Assumes:** $V > 0$; -1 $km \le h \le 47$ km
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
aerospace_engineering_ai999x.solve('aero_flight_condition', V=100.0, h=0.0)
# -> M = 0.293863, q = 6125 Pa, EAS = 100 m/s, EAS_kt = 194.384, a = 340.294 m/s, rho = 1.225
kg/m^3   (SOLVED)
```

```
aerospace_engineering_ai999x.solve('aero_flight_condition', V=230.0, h=11000.0, L=4.0)
# -> M = 0.779255, q = 9649 Pa, EAS = 125.513 m/s, EAS_kt = 243.978, a = 295.154 m/s, rho =
0.364802 kg/m^3, Re = 2.35969e+07   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flight condition: V = 100 m/s, h = 0 m	SOLVED	M = 0.293863, q = 6125 Pa, EAS = 100 m/s, EAS_kt = 194.384, a = 340.294 m/s, rho = 1.225 kg/m^3
Mach number at altitude: V = 230 m/s, h = 11 km	SOLVED	M = 0.779255, q = 9649 Pa, EAS = 125.513 m/s, EAS_kt = 243.978, a = 295.154 m/s, rho = 0.364802 kg/m^3
Flight condition: V = 230 m/s, h = 90 km	INVALID_INPUT	Mach number, dynamic pressure, equivalent airspeed and Reynolds number at an altitude: the altitude must lie between -1
Flight condition: V = 230 m/s	NEEDS_INPUT	aero_flight_condition needs h (geometric altitude (standard atmosphere)). Example: solve('aero_flight_condition', V='100